

СОДЕРЖАНИЕ

Введение.....	6
1 Обзор литературы	8
1.1 Программный интерфейс TBDb	8
1.1.1 Информация о фильме.	8
1.1.2 Поиск фильма.....	9
1.2 Рекомендательная система.....	9
1.3 Платформы машинного обучения	11
1.4 Архитектура приложения.....	12
1.4.1 Клиент-серверная архитектура.	13
1.5 Обзор аналогов	13
2 Системное проектирование.....	17
2.1 Блок клиентского приложения	17
2.2 Блок сервера.....	18
2.3 Блок загрузки предпочтений.....	18
2.5 Блок локальной базы данных.....	19
2.6 Блок анализа данных.....	19
2.7 Блок обучения модели	19
2.8 Блок базы данных фильмов.....	20
2.9 Блок подготовки исходных данных	20
3 Функциональное проектирование	21
3.1 Блок анализа данных.....	21
3.1.1 Класс DataProcessor	21
3.1.2 Класс UserPreferencesAnalyzer.....	22
3.1.3 Класс RecommendationEngine	23
3.2 Блок обучения модели	24
3.2.1 Класс ModelTainer.....	25
3.2.2 Класс CollaborativeFiltering	25
3.2.3 Класс ContentBasedFiltering.....	26
3.2.4 Класс ModelEvaluator.....	27
3.2.5 Класс FeatureExtractor.....	28
3.2.6 Класс HyperParameterTuner.....	29
3.3 Блок подготовки исходных данных	30
3.3.1 Класс DataLoader	30
3.3.2 Класс MaxFeatureExtractor.....	31
3.3.3 Класс DataNormalizer.....	32
3.3.4 Класс MovieDataValidator.....	33
3.3.5 Класс MovieDataSaver.....	34
3.3.6 Класс DataTransformer	35
3.4 Блок клиентского приложения	36
3.7 Блок реляционной базы данных	37
4 Разработка программных модулей	40
4.1 Общая архитектура системы.....	40

4.2 Ключевые алгоритмы и функции	40
4.2.1 Предобработка данных	40
4.2.2 Векторизация текста и вычисление схожести	42
4.2.3 Формирование рекомендаций	43
4.2.4 Пользовательский интерфейс.....	43
7 Техничко-экономическое обоснование разработки и реализации на рынке приложения «Сервис РЕКОМЕНДАЦИЙ ФИЛЬМОВ С использованием ИИ»	53
7.1 Характеристика программного средства, разрабатываемого для реализации на рынке.....	53
7.2 Расчёт инвестиций в разработку программного средства	53
7.2.1 Расчёт зарплат на основную заработную плату разработчиков ...	53
7.2.2 Расчёт затрат на дополнительную заработную плату разработчиков.....	55
7.2.3 Расчёт отчислений на социальные нужды	55
7.2.4 Расчёт прочих расходов	55
7.2.5 Расчёт расходов на реализацию	55
7.2.6 Расчёт общей суммы затрат на разработку и реализацию	56
7.3 Расчёт экономического эффекта от реализации программного средства на рынке	56
7.4 Расчёт показателей экономической эффективности разработки и реализации программного средства на рынке	57
7.5 Вывод об экономической целесообразности реализации проектного решения	58
Заключение	59
Список использованных источников	60
Приложение А	61
Приложение Б.....	62
Приложение В.....	63

ВВЕДЕНИЕ

Данный дипломный проект посвящён разработке программного средства для рекомендаций фильмов пользователям на основе их предпочтений.

Целью дипломного проектирования является создание сервиса рекомендаций фильмов с использованием технологий искусственного интеллекта. На основе введенных данных пользователя алгоритм будет подбирать и предлагать фильмы, что обеспечит персонализированный подход к рекомендациям, улучшая опыт просмотра и помогая пользователям находить новые интересные фильмы, соответствующие их вкусам.

В соответствии с целью определены следующие задачи:

- изучить необходимую литературу;
- провести анализ аналогов;
- разработать серверную часть программы, отвечающую за обработку данных;
- разработать клиентскую часть программы для взаимодействия с пользователями;
- протестировать программу.

Практическим применением разработки сервиса рекомендаций фильмов является создание удобной платформы для пользователей, позволяющей быстро и эффективно находить новые фильмы на основе их предпочтений. Сервис улучшает пользовательский опыт, предлагая структурированную информацию о фильмах, жанрах и рейтингах, что позволяет пользователям легко ориентироваться в обширном мире кино. Кроме того, интеграция алгоритмов искусственного интеллекта обеспечивает высокую степень точности в рекомендациях, помогая пользователям открывать для себя новые интересные фильмы, которые могут не быть на поверхности.

Разработка приложения для различных типов устройств является одной из главных задач данного дипломного проекта. Это требует создания адаптивного дизайна, который будет корректно отображаться как на настольных компьютерах, так и на мобильных устройствах.

В рамках разработки проекта также будет предусмотрена возможность интеграции с различными API для получения данных о фильмах, что позволит динамически обновлять контент и рекомендации. Кроме того, создание серверной части позволит эффективно обрабатывать запросы пользователей, управлять базами данных и реализовывать алгоритмы машинного обучения для генерации персонализированных рекомендаций.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет XX% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Программный интерфейс TMDb

The Movie Database (TMDb) – это созданная сообществом база данных фильмов и телепередач. Она содержит описания большинства фильмов и обложки на русском языке, что делает ее удобной для русскоязычных пользователей. TMDb предоставляет обширную информацию о фильмах и сериалах, включая актёрский состав, рейтинги, отзывы и многое другое [1].

Основные функции и возможности TMDb:

- информация о фильмах и сериалах: содержит описание, актёрский состав, режиссёров, продюсеров, рейтинги и многое другое;
- обложки и изображения: предлагает обложки фильмов и сериалов на русском языке, а также другие изображения, связанные с кинопроизведениями;
- редактирование пользователями: позволяет добавлять, изменять и удалять информацию о фильмах и сериалах;
- API для разработчиков: позволяет интегрировать данные о фильмах и сериалах в другие приложения и сервисы;
- социальные возможности: пользователи могут создавать списки фильмов и сериалов, комментировать и обсуждать их, а также следить за обновлениями в мире кино и телевидения.

API TMDb предоставляет возможность получения данных в формате JavaScript Object Notation (JSON), что облегчает интеграцию с различными приложениями, так как JSON является нативно поддерживаемым форматом для большинства языков программирования, включая Python и JavaScript [2]. Использование JSON позволяет разработчикам легко парсить и обрабатывать данные, не прибегая к сторонним библиотекам.

Далее в данном разделе представлены структуры ответов API TMDb на различные запросы. В каждой структуре указаны названия ключей и типы значений. Если значение является опциональным, это будет отмечено в отдельном столбце таблицы. Если такой столбец отсутствует, все значения являются обязательными. Полями в данном разделе будут называться пары ключа и значения.

Типы данных также могут варьироваться в зависимости от конкретного запроса, и далее будут описаны примеры структур данных для наиболее распространенных запросов, таких как получение информации о фильмах и поисковые запросы.

1.1.1 Информация о фильме.

В ответе за запрос, связанный с информацией о фильме, полученной из базы данных TMDb, содержатся ключевые данные, такие как название, описание, жанры, актеры, дата релиза, компания производитель и рейтинги. Эти данные являются важными для понимания содержания и контекста

фильма. Структура ответа на запрос о предоставлении информации о фильме представлена в таблице 1.1. Данная информация помогает зрителям сделать осознанный выбор при просмотре, а также дает возможность глубже оценить художественные и технические аспекты произведения.

Таблица 1.1 – Структура ответа на запрос о получении информации о фильме

Ключ	Тип	Описание
id	Число	Идентификатор
title	Строка	Название
original_language	Строка	Язык
genres	Объект	Жанры
release_data	Дата	Дата выпуска
production_company	Объект	Компания производитель
runtime	Число	Продолжительность
poster_path	Строка	Ссылка на обложку фильма
budget	Число	Бюджет фильма
overview	Строка	Описание фильма
homepage	Строка	Домашняя страница
popularity	Число	Популярность фильма
status	Строка	Статус фильма
vote_average	Число	Рейтинг фильма

1.1.2 Поиск фильма.

Структура ответа за запрос поиска фильма представлена в таблице 1.2.

Таблица 1.2 – Структура ответа на поиск фильма

Ключ	Тип	Описание
page	Число	Номер страницы
results	Объект	Информация о фильме
total_page	Число	Количество страниц
total_results	Число	Количество результатов поиска

1.2 Рекомендательная система

Рекомендательные системы используют алгоритмы машинного обучения для анализа данных о пользователях и их поведении. Эти данные могут включать информацию о просмотренных фильмах, прослушанных песнях, купленных товарах или даже о времени, проведенном на определенных страницах. Основная цель рекомендательных систем – предсказать, какой контент будет наиболее интересен пользователю, и

предложить его.

Существует несколько методов, используемых в рекомендационных системах:

- коллаборативная фильтрация;
- матричная факторизация;
- контентная фильтрация;
- гибридная система.

1.2.1 Коллаборативная фильтрация.

Коллаборативная фильтрация делится на два основных типа: метод, основанный на пользователях (user-based), и метод, основанный на товарах (item-based) [3]. В методе, основанном на пользователях, система анализирует поведение пользователя и ищет других пользователей с похожими предпочтениями. Затем она рекомендует контент, который понравился этим похожим пользователям. Например, если пользователь А и пользователь В имеют схожие вкусы в фильмах, и пользователь А посмотрел фильм, который пользователь В еще не видел, то система порекомендует этот фильм пользователю В. В методе, основанном на товарах, система анализирует характеристики товаров и ищет схожие товары. Если пользователь посмотрел или купил определенный товар, система порекомендует ему похожие товары. Например, если пользователь купил книгу по искусственному интеллекту, система может порекомендовать ему другие книги на ту же тему.

1.2.2 Матричная факторизация.

Матричная факторизация – это метод, который используется для повышения точности рекомендаций [4]. Он позволяет представить данные в виде матриц, где строки представляют пользователей, а столбцы – товары. Значения в матрице отражают предпочтения пользователей к определенным товарам. Матричная факторизация разлагает эту матрицу на две матрицы меньших размерностей: матрицу пользователей и матрицу товаров. Затем она использует эти разложения для предсказания предпочтений пользователей к товарам, которые они еще не оценили. Этот метод особенно эффективен для работы с большими объемами данных и широко используется в таких платформах, как Netflix и Amazon. Например, алгоритм SVD (Singular Value Decomposition) является популярным вариантом матричной факторизации и часто применяется в рекомендательных системах.

1.2.3 Контентная фильтрация.

Контентная фильтрация – это другой важный метод, используемый в рекомендательных системах [5]. В отличие от коллаборативной фильтрации, которая основывается на поведении пользователей, контентная фильтрация анализирует характеристики контента и сопоставляет их с предпочтениями пользователя.

Контентная фильтрация использует информацию о свойствах товаров или контента для создания рекомендаций. Например, в случае с книгами это

могут быть жанр, автор, язык и т. д. Система создает профиль пользователя на основе его предпочтений и затем сравнивает его с характеристиками доступного контента, чтобы предложить наиболее подходящие варианты. Контентная фильтрация часто используется в системах, где характеристики товаров играют ключевую роль в предпочтениях пользователей. Например, в музыкальных сервисах, таких как Spotify, используются данные о жанрах, исполнителях и альбомах для создания рекомендаций. Если пользователь часто слушает рок-музыку, система будет рекомендовать ему новые рок-альбомы и исполнителей. В случае с фильмами и сериалами, такими как Netflix, система анализирует жанры, режиссеров, актеров и другие метаданные, чтобы предложить пользователю контент, который соответствует его вкусу. Например, если пользователь часто смотрит научно-фантастические фильмы, система порекомендует ему новые фильмы и сериалы в этом жанре.

Контентная фильтрация имеет свои преимущества и недостатки. Одним из главных преимуществ является то, что она не требует данных о других пользователях и может создавать рекомендации даже для новых пользователей (проблема "холодного старта"). Однако у нее есть и свои ограничения. Например, система может не учитывать, что пользователь может быть заинтересован в контенте, который отличается от его предыдущих предпочтений. Это ограничивает разнообразие рекомендаций и может привести к "эффекту пузыря", когда пользователь получает только однотипные рекомендации.

1.2.4 Гибридная система.

Для получения наилучших результатов многие платформы используют гибридные рекомендательные системы, которые комбинируют коллаборативную фильтрацию и контентную фильтрацию. Это позволяет учесть как поведение пользователей, так и характеристики контента, обеспечивая более точные и разнообразные рекомендации.

Гибридные системы могут комбинировать различные методы несколькими способами. Один из подходов заключается в последовательном применении методов. Например, система сначала использует коллаборативную фильтрацию для определения круга возможных рекомендаций, а затем применяет контентную фильтрацию для уточнения и ранжирования этих рекомендаций. Другой подход заключается в объединении результатов разных методов. Например, система может генерировать отдельные списки рекомендаций с использованием коллаборативной и контентной фильтрации, а затем объединять их с учетом весовых коэффициентов. Это позволяет учесть преимущества каждого метода и минимизировать их недостатки.

1.3 Платформы машинного обучения

TensorFlow – это открытая программная библиотека для машинного обучения, разработанная компанией Google для решения задач построения и

тренировки нейронной сети с целью автоматического нахождения и классификации образов, достигая качества человеческого восприятия. Библиотека предлагает несколько уровней абстракции, большую гибкость, быстрое выполнение обеспечивает немедленную итерацию и интуитивно понятную отладку. Для больших задач обучения ML можно использовать API стратегии распределения для распределенного обучения на различных конфигурациях оборудования без изменения определения модели.

PyTorch – это библиотека глубокого обучения, созданная на основе Python и Torch (фреймворк на основе Lua). Он обеспечивает ускорение графического процессора, динамические вычислительные графики и интуитивно понятный интерфейс для исследователей и разработчиков глубокого обучения. PyTorch использует подход «определения по выполнению», что означает, что его вычислительные графы создаются на лету, что позволяет лучше отлаживать и настраивать модель.

Scikit-learn – один из наиболее широко используемых пакетов Python для Data Science и Machine Learning. Он позволяет выполнять множество операций и предоставляет множество алгоритмов. Scikit-learn также предлагает отличную документацию о своих классах, методах и функциях, а также описание используемых алгоритмов.

Keras это библиотека, предоставляющая интерфейс Python для искусственных нейронных сетей. Keras следует рекомендациям по снижению когнитивной нагрузки: он предлагает согласованные и простые API, минимизирует количество действий пользователя, необходимых для обычных случаев использования, и предоставляет четкие и действенные сообщения об ошибках. Keras также уделяет первостепенное внимание созданию качественной документации и руководств для разработчиков.

CNTK это стандартизированный инструмент для проектирования и развития сетей разнообразных видов, применяет искусственный интеллект для работы с большими объемами данных путем глубокого обучения, использует внутреннюю память для обработки последовательностей произвольной длины. CNTK развивает неуклонное расширение, скорость и точность с качеством коммерческого уровня применения. Имеет выраженную и простую архитектуру, совместимую с популярными языками и сетями.

1.4 Архитектура приложения

Архитектура приложения – это структурное определение его компонентов и их взаимодействий, которое служит основой для разработки, поддержки и масштабирования программного обеспечения. Правильно спроектированная архитектура обеспечивает гибкость, устойчивость и возможность интеграции новых функций.

Выбор архитектуры является важной частью разработки программного обеспечения.

Существует несколько типов архитектур приложений, каждая из которых имеет свои особенности и подходит для различных сценариев

использования. К основным типам относятся:

- монолитная архитектура;
- микросервисная архитектура;
- клиент-серверная архитектура;
- сервисная архитектура;
- архитектура на основе событий;
- архитектура с использованием очередей сообщений.

Каждый тип архитектур имеет свои сильные и слабые стороны, и выбор подходящей архитектуры зависит от требований проекта, его масштабируемости, сложности и других факторов. Среди существующих архитектур, клиент-серверная архитектура выделяется своим четким разделением задач и возможностью использования разных технологий на клиенте и сервере, что делает её особенно подходящей для поставленных в проекте задач.

1.4.1 Клиент-серверная архитектура.

Клиент-серверная архитектура позволяет четко разделить обязанности между клиентской и серверной частями. Это упрощает процесс разработки, так как команды могут сосредоточиться на своих областях, используя различные технологии и инструменты. Клиент, отвечающий за взаимодействие с пользователем, может быть разработан с учетом удобства и отзывчивости, в то время как серверная часть будет сосредоточена на обработке бизнес-логики и управлении данными.

Кроме того, данный подход обеспечивает гибкость в масштабировании. По мере роста нагрузки на приложение есть возможность независимо масштабировать серверную часть, добавляя новые ресурсы, не затрагивая клиентскую логику. Это особенно важно в условиях динамично меняющихся требований и увеличивающегося числа пользователей.

Также клиент-серверная архитектура предоставляет возможности для улучшения безопасности. Сервер может централизованно управлять аутентификацией и авторизацией пользователей, что минимизирует риски, связанные с утечкой данных.

Таким образом, учитывая все вышеперечисленные преимущества, клиент-серверная архитектура является оптимальным выбором для данного проекта. Она обеспечит необходимую гибкость, производительность и безопасность, что позволит успешно реализовать задуманное и адаптироваться к будущим изменениям.

1.5 Обзор аналогов

Существуют различные сервисы и платформы с рекомендательными системами фильмов со своими исключительными особенностями и подходом к расположению пользователей. На сегодняшний день, самыми популярными являются следующие рекомендательные сервисы: Netflix, Кинопоиск, MovieTon.

1.5.1 Netflix.

Netflix — это стриминговый сервис, позволяющий смотреть разнообразные удостоенные наград фильмы, сериалы, аниме, документальные фильмы и многое другое на тысячах устройств с подключением к интернету. Когда пользователь использует сервис Netflix, система рекомендаций пытается помочь ему найти фильм или сериал, которые будет интересно смотреть. Вероятность того, что пользователю понравятся конкретные фильм или сериал, оценивается на основе множества факторов. Учитываются особенности взаимодействия с сервисом (например, история просмотра), выбор других пользователей сервиса с похожими вкусами и предпочтениями, а также характеристики самого фильма или сериала: жанр, категория, актеры, год выхода и другое [6].

Помимо информации о том, что именно пользователь смотрит на Netflix, сервис также использует данные о том, как это делает пользователь: в какое время суток, на каких языках, с каких устройств и как долго.

Система рекомендаций ранжирует фильмы, сериалы и игры и размещает их на пользовательской стартовой странице Netflix так, чтобы пользователи быстрее могли найти для себя подходящую ленту.

На каждой странице есть несколько уровней персонализации. Например, если говорить о рядах, персонализация затрагивает следующее: ряд (например, «Вы не досмотрели»), набор видео для заполнения ряда и порядок этих видео.

Видео в каждом ряду отсортированы от наиболее подходящего до наименее подходящего, слева направо. Если пользователь выбрал в качестве основного языка сервиса арабский или иврит, то рекомендации будут отсортированы справа налево.

Когда пользователи смотрят Netflix, сервис регистрирует, какие видео начали смотреть, какие досмотрели и как их оценили, и постоянно переобучает алгоритмы в соответствии с этими новыми данными, чтобы более точно предсказывать, что может понравиться пользователям.

Интерфейс сервиса представлен на рисунке 1.1.

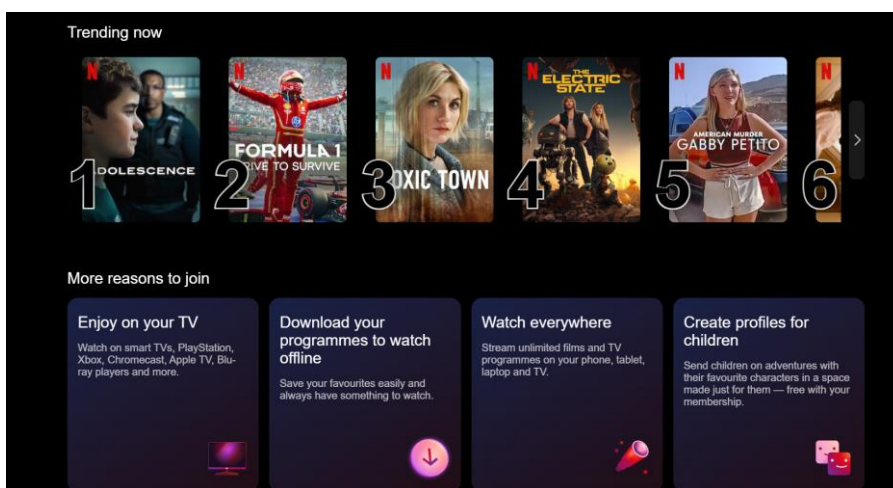


Рисунок 1.1 – Интерфейс сервиса Netflix

1.5.2 Кинопоиск.

Для предоставления наиболее подходящих релевантных и точных тайтлов сервис собирает информацию о пользователях [7]. Тайтлами называются единицы контента в сервисе – фильмы, сериалы, телеканалы и прочее.

Кинопоиск учитывает следующие данные:

- поисковые запросы;
- историю поиска;
- историю просмотров;
- оценки тайтлов;
- добавление тайтлов в коллекцию.

Сервис анализирует данные, о которых речь выше, а также жанр, название, сюжет, актеров, режиссеров, получение фильмом тех или иных наград и премий, историю взаимодействия пользователей с тайтлами, историю просмотров – и на основе этого предлагает пользователю тот или иной тайтл – фильм, подборку по теме, информацию об актере и тп. Алгоритм использует в работе машинное обучение и нейросети, а также так называемые матричные факторизации – они основаны на множестве видов пользовательского фидбека и разметки контента.

Ранжирование собирается таким образом, чтобы сделать метрику качества рекомендаций максимально подходящей для интересов пользователя. Эта метрика показывает, как часто пользователь обращается к кино и насколько долго его смотрит. Результаты подбираются таким образом, чтобы поощрить время просмотра в течение дня (и увеличить количество дней с просмотром в течение недели). Алгоритм расчета и параметры метрики качества рекомендаций подбираются так, чтобы максимально хорошо соответствовать метрике «интересности сервиса». Правда детального разбора о том, как вычисляется эта метрика, сервис не предоставляет.

Интерфейс данного приложения представлен на рисунке ниже:

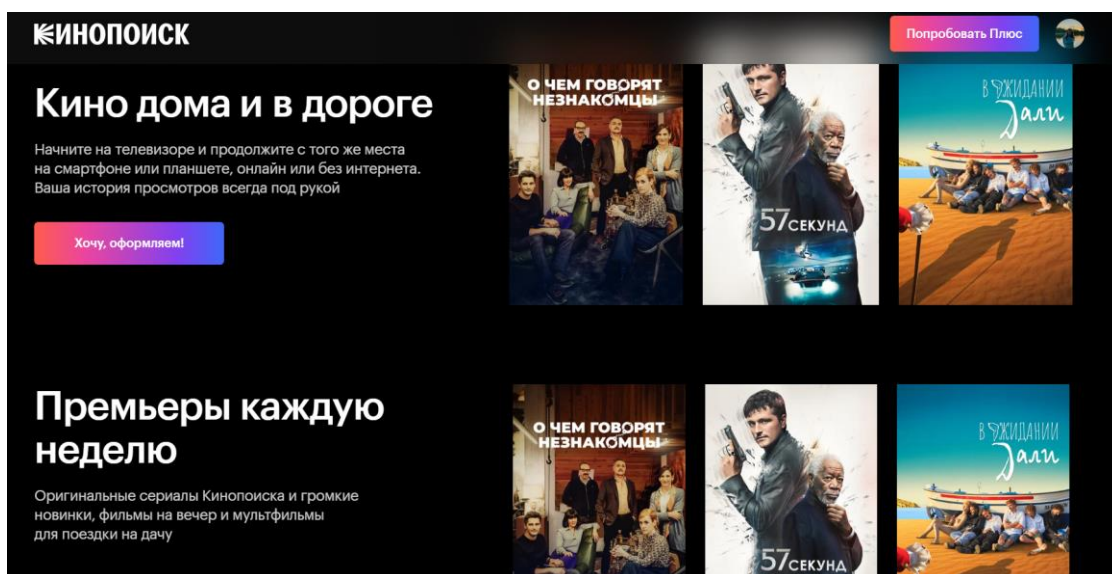


Рисунок 1.2 – Интерфейс платформы Кинопоиск

1.5.3 MovieTon.

Главная задача и особенность Movieton – определение интересов его пользователей (посредством мини-тестирования) и формирование на базе полученных данных максимально подходящей для каждого конкретного человека подборки фильмов [8]. Всего несколько стандартных вопросов, размещенных на сайте, полученные на них ответы, позволяют умной системе производить быструю фильтрацию коллекций, моментальную выдачу кинокартин по сугубо личным интересам пользователя ресурса.

С целью подбора наиболее подходящей картины, необходимо кликнуть на кнопку «Подобрать фильм». Далее ответить на вопросы системы и получить список кинолент по заданным критериям. С тем чтобы найти случайную ленту, следует жать на кнопку «Случайный фильм».

Также на сайте возможен и самостоятельный поиск картин. Для удобства пользователей кинокартины сгруппированы по подборкам и актерам. После выбора того или иного пути поиска открывается доступ к обширным коллекциям фильмов. Заглянув на персональную страничку выбранного кино, можно ознакомиться с афишей ленты, кратким описанием сюжета, артистами, сыгравшими главные и второстепенные роли, трейлером картины, ее годом выпуска, продолжительностью сеанса.

Сервис удобен в навигации, имеет наглядную подачу материала. Разработчиками ресурса выбран интерактивный формат донесения важной информации о кино потенциальным зрителям, что делает времяпрепровождение на сайте очень комфортным и приятным.

Пройдя регистрацию, каждый желающий может воспользоваться не только удобным функционалом сайта «MovieTon», оценить все достоинства моментального поиска фильмов, сериалов, но и оставлять отзывы о просмотренных кинолентах, добавлять понравившиеся картины в Избранное.

Интерфейс приложения представлен на рисунке 1.3.

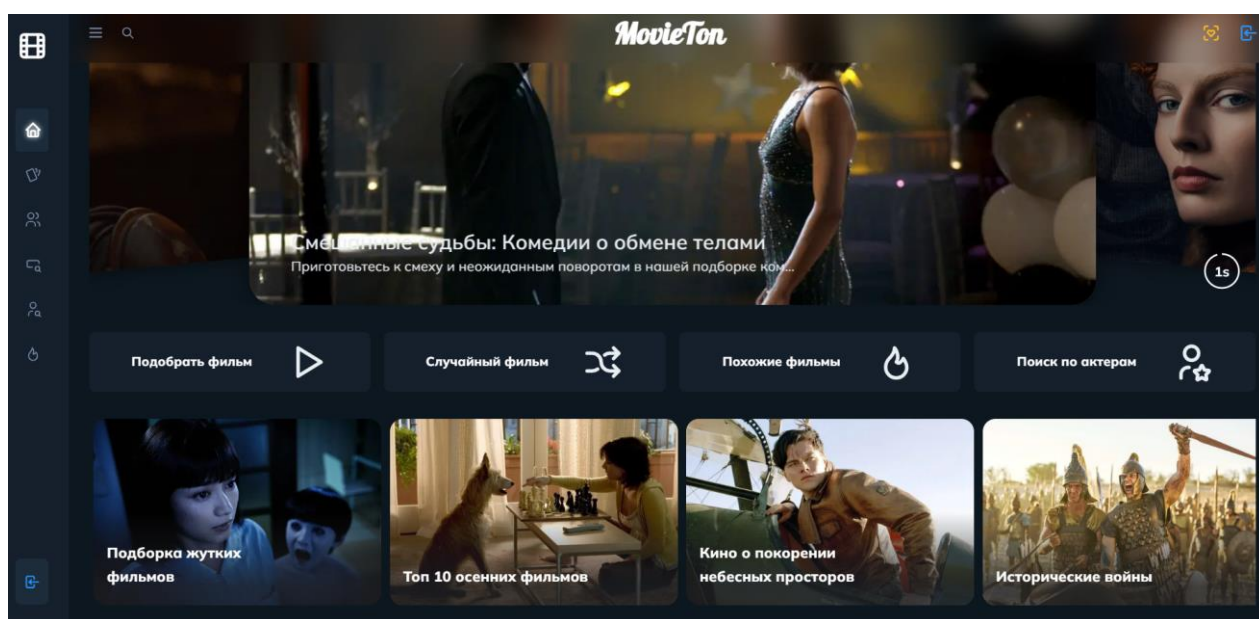


Рисунок 1.3 – Интерфейс сервиса MovieTon

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

Перед тем, как приступить к разработке приложения для рекомендаций фильмов необходимо выделить определенную структуру проекта. Для эффективной работы сервиса можно выделить несколько блоков, каждый из которых отвечает за свою функцию. Перечень основных компонентов, которые будут формировать архитектуру разрабатываемого приложения:

- блок клиентского приложения;
- блок сервера;
- блок загрузки предпочтений;
- блок локальной базы данных;
- блок анализа данных;
- блок обучения модели;
- блок базы данных фильмов;
- блок подготовки исходных данных.

Подбор персональных рекомендаций будет происходить следующим образом:

- пользователь вводит предпочтения через клиентское приложение;
- клиентское приложение отправляет данные на сервер;
- сервер передает данные в блок загрузки предпочтений;
- блок загрузки предпочтений отправляет данные в блок анализа данных;
- блок анализа данных использует блок машинного обучения, которая анализирует предпочтения и генерирует рекомендации;
- рекомендации возвращаются на сервер;
- сервер отправляет рекомендации в клиентское приложение;
- клиентское приложение отображает рекомендации пользователю;
- блок обучения модели обновляет модель на основе новых пользовательских данных, полученных из локальной базы данных или базы данных фильмов.

Взаимосвязь между блоками проекта отражена на структурной схеме ГУИР.400201.043 С1

2.1 Блок клиентского приложения

Этот блок представляет собой удобный интерфейс пользователю. Он отвечает за отображение информации и обработку пользовательских действий.

Интерфейс приложения представляет собой совокупность визуальных элементов, таких как кнопки, меню, поля ввода, и списки, которые позволяют пользователю взаимодействовать с приложением. Дизайн интерфейса должен быть интуитивно понятным и удобным, что позволит легко находить фильмы и сохранять предпочтения.

Основными компонентами интерфейса являются форма ввода

предпочтений и отображение рекомендаций. Форма ввода представляет собой карточку с незаполненными постером и полем с названием, чтобы при последующем вводе в поле название предпочитаемого фильма, пользователь мог выбрать необходимый фильм и выпадающего списка, после чего установится название и постер выбранного фильма. На основе введенного пользователем фильма происходит отображение списка рекомендаций, который представлен в виде карточек с информацией рекомендуемых фильмов: описанием, постером, жанрами, актерами и прочее.

Этот блок непосредственно взаимодействует с сервером для отправки и получения данных.

2.2 Блок сервера

Блок сервера является центральным компонентом архитектуры приложения и отвечает за обработку запросов от клиентского приложения, управление данными и взаимодействие с другими модулями системы. Он принимает входящие данные от клиента, такие как предпочтения пользователей, и обрабатывает их, используя бизнес-логику и алгоритмы для генерации рекомендаций.

Сервер управляет связью с базами данных, обеспечивая хранение, извлечение и обновление информации о фильмах и предпочтениях пользователей. Он также осуществляет вызовы к модулям машинного обучения, передавая им данные для анализа и получения рекомендаций. После обработки данных сервер формирует ответ и отправляет его обратно на клиент, где результаты отображаются пользователю.

Кроме того, сервер отвечает за безопасность приложения, включая аутентификацию пользователей и защиту данных. Он может также реализовывать механизмы кэширования для ускорения доступа к часто запрашиваемым данным, что улучшает производительность приложения. В целом, блок сервера обеспечивает надежную и эффективную работу всего приложения, координируя взаимодействие между клиентом, базами данных и модулями анализа данных.

2.3 Блок загрузки предпочтений

Этот блок является компонентом серверной части приложения, отвечающий за прием и обработку пользовательских данных о предпочтениях в фильмах. Когда пользователь вводит свои предпочтения через клиентское приложение, этот блок получает эти данные и подготавливает их для дальнейшей обработки.

Он выполняет валидацию входящих данных, чтобы убедиться, что они корректны и соответствуют ожидаемым форматам. После проверки блок загружает предпочтения в соответствующий формат, который может быть использован в алгоритмах анализа данных. Затем он передает эти данные в модуль анализа, который использует их для формирования рекомендаций.

Этот компонент отвечает за установление связи между пользовательским вводом и алгоритмами, обеспечивающими персонализированный опыт.

2.5 Блок локальной базы данных

Блок локальной базы данных представляет собой компонент, который отвечает за хранение и управление данными о фильмах, метаданными и пользовательскими данными в пределах приложения. Эта база данных обеспечивает быстрый доступ к информации, что критически важно для эффективной работы системы рекомендаций.

Локальная база данных хранит различные данные, включая названия фильмов, жанры, описания, рейтинги и обложки, а также информацию о пользовательских данных, а также их оценках и предпочтениях. Она позволяет быстро извлекать данные при выполнении запросов от клиентского приложения или других модулей, что значительно ускоряет процессы поиска и генерации рекомендаций.

Блок локальной базы данных также обеспечивает возможности для обновления информации, например, при добавлении новых фильмов или изменении пользовательских предпочтений.

2.6 Блок анализа данных

Блок анализа данных отвечает за обработку и интерпретацию пользовательских предпочтений с целью генерации персонализированных рекомендаций. Этот блок использует алгоритмы машинного обучения и статистические методы для анализа собранных данных о фильмах и пользовательских оценках.

Когда блок загрузки предпочтений передает данные о предпочтениях пользователя, блок анализа начинает обрабатывать эту информацию, выявляя паттерны и зависимости. Он учитывает различные факторы, такие как жанры, рейтинги и другие характеристики фильмов, чтобы определить, какие фильмы могут быть интересны конкретному пользователю.

В итоге, этот компонент обеспечивает основу для создания персонализированного опыта пользователя, позволяя системе адаптироваться к изменениям в предпочтениях и предоставлять рекомендации, которые соответствуют индивидуальным вкусам.

2.7 Блок обучения модели

Блок обучения модели является важным компонентом системы, он отвечает за создание и оптимизацию алгоритмов машинного обучения, используемых для генерации рекомендаций. Этот блок принимает данные о предпочтениях пользователей и характеристиках фильмов, чтобы обучить модель на основе этих данных.

Процесс обучения включает в себя несколько этапов: выбор алгоритма, подготовка данных, обучение модели и оценка её производительности. На первом этапе выбирается подходящий алгоритм, такой как коллаборативная фильтрация, контентная фильтрация или гибридные методы. Затем данные подготавливаются, включая очистку, нормализацию и разделение на обучающую и тестовую выборки.

После этого модель обучается на обучающей выборке, где она анализирует паттерны и взаимосвязи в данных. Затем происходит тестирование модели на тестовой выборке для оценки её точности и способности делать предсказания.

2.8 Блок базы данных фильмов

Этот блок представляет собой компонент, который отвечает за хранение и управление информацией о фильмах, полученной из TMDb (The Movie Database). Блок включает в себя разнообразные данные: названия, жанры, описания, рейтинги, даты выхода и обложки, которые предоставляются API TMDb.

База данных фильмов обеспечивает быстрый доступ к информации при выполнении запросов от других компонентов системы, таких как блок поиска фильмов и блок анализа данных. Она поддерживает различные операции, включая извлечение данных о фильмах, обновление информации при изменениях на стороне TMDb и синхронизацию данных.

Этот компонент также обеспечивает целостность и согласованность данных, что позволяет системе функционировать эффективно и предоставлять пользователю актуальную информацию о фильмах.

2.9 Блок подготовки исходных данных

Блок подготовки исходных данных отвечает за обработку и трансформацию необработанных данных о фильмах перед их использованием в других компонентах системы, таких как блок анализа данных или блок обучения модели. Процесс начинается с извлечения необходимой информации о фильмах из различных источников, таких как API TMDb, базы данных или другие внешние ресурсы.

После этого происходит очистка данных, которая включает в себя удаление дубликатов, исправление ошибок и недостающих значений, а также фильтрацию неактуальной или нерелевантной информации. Затем данные преобразуются в удобный для анализа формат, включая нормализацию значений и создание новых переменных.

Блок подготовки исходных данных обеспечивает высокое качество и релевантность информации, используемой для анализа, обучения моделей и генерации рекомендаций. Это, в свою очередь, значительно улучшает точность и эффективность работы остальных компонентов системы, что критически важно для достижения успешных результатов.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

В этом разделе пояснительной записки мы подробно рассмотрим функционирование программного обеспечения. Для достижения целей проекта будет представлен детальный обзор архитектуры реализуемого решения, а также проанализированы основные классы, их структура, взаимосвязи и зависимости. Мы также рассмотрим данные таблиц, используемых в базах данных, а также методы классов, поля и константы.

Поскольку в разработке приложения применяется объектно-ориентированная парадигма программирования, все модули программного обеспечения представлены в виде различных классов, логически сгруппированных в зависимости от их функциональности. На диаграмме ГУИР.400201.043 РР.1 показана структура классов данного программного обеспечения.

Основным блоком разрабатываемого программного продукта является блок анализа данных, который представляет собой систему рекомендаций фильмов на основе искусственного интеллекта. Этот блок выполняет несколько ключевых функций.

Во-первых, он осуществляет предварительную обработку данных о фильмах. Во-вторых, блок использует алгоритмы машинного обучения для анализа предпочтений пользователей

Наконец, он генерирует рекомендации, основываясь на внешних атрибутах фильмов, таких как жанр, актерский состав и рейтинги.

Остальные компоненты, которые составляют функциональную структуру приложения, перечислены ниже:

- блок клиентского приложения;
- блок реляционной базы данных;
- блок обучения модели;
- блок подготовки исходных данных.

3.1 Блок анализа данных

Блок анализа данных отвечает за следующие задачи:

- подготовка данных;
- анализ предпочтений пользователей;
- генерация рекомендаций;
- анализирует данные пользователя для улучшения рекомендаций.

Для реализации перечисленных выше функций предусмотрены классы, отвечающие за отдельно взятые задачи. Подробное описание классов рассмотрено далее.

3.1.1 Класс `DataProcessor`

Класс `DataProcessor` является важным компонентом системы анализа данных, отвечающим за предварительную обработку сырых данных. Цель этого класса – подготовить данные для дальнейшего анализа или обучения

моделей машинного обучения. Чистые и хорошо структурированные данные являются основой для получения качественных и точных результатов, поэтому этот класс выполняет ряд ключевых функций.

Основные поля и функции класса:

1 `clean_data(self)`. Функция очистки данных. Очистка данных включает в себя удаление дубликатов, которые могут исказить результаты анализа. Дубликаты могут возникать по разным причинам, например, при сборе данных из разных источников или в результате ошибки при вводе данных. Кроме того, класс обрабатывает пропуски в данных, заполняя их средними значениями или другими статистическими показателями, что позволяет избежать потери информации.

2 `normalize_data(self)`. Нормализует оценочные значения в диапазоне от 0 до 1. Нормализация – это процесс приведения данных к единому масштабу, что особенно важно, когда разные признаки имеют различные диапазоны значений. Нормализация помогает улучшить производительность алгоритмов машинного обучения, обеспечивая более корректное сравнение и анализ.

3 `remove_outliers(self)`. Удаляет выбросы из данных с использованием метода IQR. Выбросы могут значительно ухудшить качество модели, поскольку они могут искажать статистические характеристики данных. Класс использует методы, такие как межквартильный размах (IQR), для выявления и удаления выбросов. Это позволяет сосредоточиться на основном тренде данных и улучшить результаты анализа.

4 `transform_data(self)`. Выполняет дополнительные преобразования данных. Часто данные содержат категориальные переменные (например, жанр фильма), которые не могут быть напрямую использованы в алгоритмах машинного обучения. Класс `DataProcessor` включает методы для преобразования таких переменных в числовой формат, что делает их пригодными для дальнейшего анализа. Одним из распространенных методов является one-hot кодирование, которое создает бинарные столбцы для каждой категории.

3.1.2 Класс `UserPreferencesAnalyzer`

Класс `UserPreferencesAnalyzer` предоставляет методы для анализа и управления предпочтениями пользователей. Этот класс помогает выявить индивидуальные предпочтения пользователей на основе их взаимодействий с контентом, таких как оценки, просмотры и другие действия. Понимание предпочтений пользователей позволяет создавать более точные и персонализированные рекомендации, что повышает удовлетворенность пользователей и увеличивает вовлеченность.

Основные функции класса:

1 `get_user_preferences(user_id)`. Метод извлекает предпочтения конкретного пользователя из базы данных. Он возвращает информацию о фильмах, которые пользователь оценил, а также их рейтинг и жанры.

2 `update_preferences(user_id, movie_id, rating)`. Метод обновляет предпочтения пользователя, сохраняя новую оценку для конкретного фильма. Обновление предпочтений критически важно для динамического подхода к рекомендациям. Каждое взаимодействие пользователя с контентом может влиять на его будущие рекомендации, если пользователь оценил новый фильм, система обновляет данные, чтобы учесть эту информацию в дальнейшем.

3 `analyze_trending_preferences()`. Этот метод анализирует общие предпочтения пользователей и выявляет тренды. Если определенный фильм становится популярным, система будет рекомендовать его всем пользователям.

4 `recommend_similar_movies(user_id)`. Метод генерирует список фильмов, похожих на те, которые пользователь оценил высоко. Этот метод является основой для рекомендаций. Если пользователь хочет подобрать фильм на основе нескольких других, то метод будет предлагать наиболее релевантные фильмы, которые будут схожи с перечисленными описанием, жанрами, актерами и режиссерами. Это поможет удержать пользователей, предлагая им контент, который соответствует их вкусам.

5 `collect_user_feedback(user_id, movie_id, feedback)`. Собирает обратную связь от пользователей относительно рекомендованного контента. Обратная связь от пользователей важна для улучшения алгоритмов рекомендаций. Если система получает много негативной обратной связи по определенному фильму, это сигнализирует о необходимости пересмотра рекомендаций для данного пользователя или группы пользователей.

3.1.3 Класс `RecommendationEngine`

Класс `RecommendationEngine` обеспечивает генерацию рекомендаций на основе анализа предпочтений пользователей и характеристик контента. Он позволяет пользователям получать персонализированные рекомендации фильмов, что улучшает их взаимодействие с платформой и удовлетворенность. В классе используются различные алгоритмы и методы, чтобы создать список фильмов, которые, интересуют пользователя, основываясь на их предпочтениях.

Основные функции класса:

1 `generate_recommendations(user_id)`. Этот метод создает список рекомендуемых фильмов для конкретного пользователя на основе их предпочтений и оценок. Он анализирует данные о фильмах, которые пользователь оценил высоко, и использует алгоритмы, такие как коллаборативная фильтрация, чтобы найти фильмы, которые могут его заинтересовать. В результате выполнения функции будет список, содержащий фильмы, наиболее подходящие пользователю.

2 `filter_recommendations(recommendations)`. Метод фильтрует сгенерированные рекомендации по заданным критериям, таким как жанр, год выпуска или рейтинг. Это позволяет пользователю получить более

целенаправленные рекомендации, которые соответствуют его предпочтениям.

3 `update_recommendation_model()`. Этот метод обновляет модель рекомендаций на основе новых данных, таких как свежие оценки пользователей или новые поступления фильмов в библиотеку. Обновление модели необходима для того, чтобы система рекомендаций была актуальной.

4 `evaluate_recommendations(user_id, recommended_movies)`. Метод оценивает качество предоставленных рекомендаций, анализируя, насколько они соответствуют фактическим предпочтениям пользователя. Он использует метрики, такие как точность, полнота и F1-мера, чтобы оценить, насколько хорошо система предсказывает интересы пользователя на основе его прошлого поведения. Это позволяет улучшить алгоритмы рекомендаций и повысить их точность.

5 `get_popular_movies()`. Этот метод возвращает список популярных фильмов на платформе, основываясь на общих оценках пользователей. Эти данные могут быть полезны для создания рекомендаций для новых пользователей или для анализа трендов.

Благодаря функциональности данного класса пользователи могут быстро находить интересующий их контент, что увеличивает время их взаимодействия с платформой и способствует повышению уровня удовлетворенности. Эффективная система рекомендаций способствует увеличению прибыли, так как пользователи, получающие релевантные рекомендации, с большей вероятностью будут возвращаться на платформу.

3.2 Блок обучения модели

Блок обучения модели является важной частью проекта, так как он отвечает за разработку и оптимизацию алгоритмов, которые генерируют рекомендации для пользователей. Этот блок включает в себя несколько ключевых классов, обеспечивающих создание, обучение и оценку моделей на основе данных о пользователях и фильмах.

Он реализует методы коллаборативной и контентной фильтрации, обучаясь на исторических данных о предпочтениях пользователей. Процесс включает извлечение и подготовку признаков, таких как жанры фильмов и актерский состав, что позволяет моделям эффективно предсказывать интересы пользователей.

После обучения модели проходят тщательную оценку с использованием метрик, таких как RMSE и MAE, что позволяет точно определить качество рекомендаций. Настройка гиперпараметров, включая число соседей и коэффициенты регуляризации, обеспечивает максимальную производительность моделей. Кроме того, блок гарантирует возможность сохранения и загрузки обученных моделей, что существенно экономит время на повторное обучение.

Регулярное обновление моделей с использованием новых данных позволяет системе поддерживать актуальность рекомендаций, учитывая изменяющиеся предпочтения пользователей и новые поступления контента.

Классы блока подробно описаны далее.

3.2.1 Класс `ModelTrainer`

Класс `ModelTrainer` играет центральную роль в блоке обучения модели. Он отвечает за обучение и оптимизацию алгоритмов, которые используются для генерации рекомендаций на основе данных о пользователях и фильмах. Благодаря своей архитектуре и функциональности, этот класс обеспечивает эффективное взаимодействие с различными моделями и алгоритмами, позволяя системе адаптироваться к меняющимся требованиям и предпочтениям пользователей.

Класс включает в себя несколько ключевых функций:

1 `train_model(training_data)`. Обучает модель на предоставленных данных, используя различные алгоритмы машинного обучения. Этот метод принимает матрицу предпочтений пользователей и фильмов, что позволяет извлекать важные паттерны и зависимости.

2 `save_model(file_path)`. Сохраняет обученную модель в указанный файл, что позволяет избежать повторного обучения и значительно экономит ресурсы. Это особенно важно в условиях реального времени, где скорость обработки запросов критична.

3 `load_model(file_path)`. Загружает ранее сохраненную модель, что позволяет быстро восстановить состояние системы без необходимости повторного обучения. Это обеспечивает гибкость и масштабируемость применения модели в будущем.

4 `evaluate_model(test_data)`. Оценивает качество модели на тестовых данных, используя различные метрики, такие как RMSE и MAE. Это позволяет определить, насколько точно модель предсказывает интересы пользователей, и выявить области для улучшения.

5 `update_model(new_data)`. Обновляет существующую модель новыми данными, что позволяет поддерживать актуальность рекомендаций. Это особенно важно в быстро меняющейся среде, где предпочтения пользователей могут изменяться в зависимости от новых трендов и контента.

3.2.2 Класс `CollaborativeFiltering`

Класс `CollaborativeFiltering` реализует один из наиболее популярных подходов к построению рекомендаций — коллаборативную фильтрацию. Этот метод основывается на анализе предпочтений пользователей, предполагая, что пользователи, которые согласны в своих оценках определенных объектов, будут согласны и в своих оценках других объектов. Класс обеспечивает обучение и предсказание на основе взаимодействий пользователей с фильмами, что позволяет эффективно генерировать рекомендации.

Класс включает следующие ключевые функции:

1 `fit(user_item_matrix)`. Этот метод принимает матрицу предпочтений пользователей и фильмов, где строки представляют

пользователей, а столбцы — фильмы. Метод использует алгоритмы, такие как сингулярное разложение матриц (SVD) или метод ближайших соседей, для выявления скрытых паттернов в данных. Например, после применения SVD модель может выявить, что пользователи, которые любят комедии, также часто оценивают высоко романтические фильмы. Таким образом, `fit` не только обучает модель, но и создает основу для последующих предсказаний.

2 `get_recommendations(user_id)`. Этот метод генерирует список рекомендаций для конкретного пользователя, основываясь на его предыдущих оценках и предпочтениях. Он анализирует, какие фильмы получили высокие оценки от пользователей, схожих с данным пользователем, и формирует список фильмов, которые, вероятно, ему понравятся. Например, если пользователь С оценил несколько фильмов высоко, и эти фильмы также понравились пользователям D и E, которые в свою очередь оценили фильм Y, то `get_recommendations` может предложить фильм Y пользователю С.

3 `evaluate(test_data)`. Метод для оценки точности модели на тестовых данных. Он сравнивает предсказанные рейтинги с фактическими оценками пользователей, используя такие метрики, как RMSE или MAE. Например, если модель предсказала рейтинг 4 для фильма, который фактически получил 5, это будет учтено в оценке. Это позволяет разработчикам понять, насколько хорошо модель справляется с задачами предсказания и какие аспекты требуют улучшения.

4 `update(user_item_matrix)`. Этот метод позволяет обновлять модель новыми данными о предпочтениях пользователей и фильмов. При добавлении новых пользователей или фильмов в систему важно, чтобы модель могла адаптироваться, сохраняя актуальность рекомендаций. Например, если новый пользователь начинает оценивать фильмы, метод `update` интегрирует его данные, позволяя системе быстро адаптироваться к новым предпочтениям.

3.2.3 Класс `ContentBasedFiltering`

Класс `ContentBasedFiltering` реализует метод контентной фильтрации, который основывается на характеристиках фильмов. Вместо того чтобы полагаться на предпочтения других пользователей, данный подход анализирует атрибуты фильмов и предпочтения самого пользователя. Это позволяет системе рекомендовать контент, схожий с тем, что пользователь уже оценил высоко. Класс осуществляет обучение и предсказания, основываясь на контентных признаках, что делает его особенно полезным в ситуациях, когда данные о пользователях ограничены.

Функции класса представлены ниже:

1 `fit(movie_features)`. Метод принимает матрицу признаков фильмов, где строки представляют фильмы, а столбцы — различные характеристики, такие как жанр, режиссер, актеры и ключевые слова. Этот метод создает профиль контента для каждого фильма, что позволяет модели выявить схожие фильмы. Например, если фильм А принадлежит к жанру "драма" и имеет в актерском составе известного актера, то `fit` сможет выделить другие фильмы с похожими признаками и создать базу для дальнейших

рекомендаций.

2 `predict(user_preferences)`. Метод используется для предсказания рейтингов фильмов на основе предпочтений пользователя. Он анализирует, какие фильмы пользователь оценил высоко, и сравнивает их характеристики с фильмами, которые еще не были просмотрены. Например, если пользователь предпочитает фильмы с элементами научной фантастики и высокими рейтингами, `predict` предложит фильмы, которые имеют схожие жанры и элементы. Это позволяет модели адаптироваться к индивидуальным интересам пользователя.

3 `get_recommendations(user_id)`. Этот метод генерирует список рекомендаций, основываясь на профиле предпочтений конкретного пользователя. Он анализирует фильмы, которые пользователь оценил высоко, и ищет другие фильмы с похожими характеристиками. Например, если пользователь оценил высоко несколько триллеров с определенным актером, `get_recommendations` сможет предложить другие триллеры с тем же актером или в том же стиле, что делает рекомендации более персонализированными.

4 `evaluate(test_data)`. Метод оценивает качество рекомендаций на тестовых данных, сравнивая предсказанные рейтинги с фактическими оценками. Используя метрики, такие как Precision и Recall, он позволяет понять, насколько точно система предсказывает интересы пользователя. Например, если модель предложила фильм, который пользователь оценил на 5, это будет учитываться в оценке, что дает разработчикам возможность контролировать и улучшать качество рекомендаций.

5 `update(movie_features)`. Этот метод позволяет обновлять модель новыми данными о фильмах и их характеристиках. При добавлении новых фильмов в каталог важно, чтобы система могла адаптироваться и включать их в процесс генерации рекомендаций. Например, если новый фильм с известным актером или в новом жанре добавляется в базу данных, метод `update` обновит модель, обеспечивая актуальность рекомендаций.

3.2.4 Класс `ModelEvaluator`

Класс `ModelEvaluator` отвечает за оценку качества обученных моделей, обеспечивая использование различных метрик для анализа их производительности. Этот класс позволяет получить глубокое понимание того, насколько хорошо модели справляются с задачами предсказания, и выявить области, требующие улучшения.

Основные функции класса:

1 `evaluate_model(test_data)`. Метод принимает тестовые данные и выполняет оценку модели, сравнивая предсказанные рейтинги с фактическими оценками пользователей. Он использует метрики, такие как RMSE (корень из среднеквадратичной ошибки) и MAE (средняя абсолютная ошибка), чтобы количественно оценить точность предсказаний. Например, если модель предсказывает рейтинг 4 для фильма, который на самом деле получил 5, это будет учтено в расчете метрик.

2 `cross_validate(folds)`. Этот метод осуществляет кросс-валидацию модели, разбивая данные на несколько частей (фолдов) и обучая модель на каждой из них, оценивая ее на оставшихся данных. Это позволяет оценить стабильность и обобщающую способность модели. Например, если модель показывает высокие результаты на разных фолдах, это свидетельствует о ее надежности и способности к обобщению.

3 `log_evaluation_results(results)`. Метод записывает результаты оценки модели в журнал, включая метрики, такие как RMSE и MAE, а также параметры модели. Это позволяет отслеживать изменения производительности модели с течением времени и служит основой для анализа эффективности различных подходов.

4 `visualize_metrics(metrics)`. Этот метод генерирует визуализации для метрик оценки, таких как графики изменения RMSE и MAE в зависимости от гиперпараметров.

5 `generate_report(metrics)`. Метод создает отчет по результатам оценки, включающий основные метрики, графики и рекомендации по улучшению модели.

3.2.5 Класс `FeatureExtractor`

Класс `FeatureExtractor` подготавливает данные для алгоритмов машинного обучения, извлекая и обрабатывая признаки, которые используются для обучения моделей. Этот класс обеспечивает преобразование сырых данных в формат, удобный для анализа, что является важным шагом для повышения точности рекомендаций.

Класс содержит следующие функции:

1 `extract_features(movie_data)`. Метод принимает сырые данные о фильмах и извлекает ключевые признаки, такие как жанр, актерский состав, режиссер и другие атрибуты. Например, он может преобразовать текстовые описания фильмов в векторные представления, что позволяет модели лучше понимать содержание. Это создает основу для контентной фильтрации и улучшает качество рекомендаций.

2 `normalize_features(features)`. Этот метод нормализует извлеченные признаки, приводя их к единому масштабу. Нормализация важна для предотвращения ситуации, когда признаки с большим масштабом доминируют над другими. Например, если один признак варьируется от 1 до 100, а другой — от 0 до 1, нормализация обеспечит равный вес для всех признаков, что в свою очередь повысит точность предсказаний.

3 `encode_categorical_features(features)`. Метод занимается обработкой категориальных признаков, таких как жанры, имена актеров и режиссеров, преобразуя их в числовой формат. Это может включать использование методов, таких как one-hot encoding, что позволяет моделям работать с категориальными данными. Например, жанр "комедия" может быть представлен как [1, 0, 0] в случае трех жанров, что делает данные пригодными для алгоритмов машинного обучения.

4 `reduce_dimensionality(features)`. Этот метод уменьшает размерность извлеченных признаков, используя техники, такие как РСА (анализ главных компонент). Уменьшение размерности помогает снизить вычислительные затраты и предотвратить переобучение модели. Например, если у вас есть 100 признаков, метод может сократить их до 20, сохраняя при этом наиболее важную информацию.

5 `generate_feature_matrix(user_movie_interactions)`. Метод создает матрицу признаков на основе взаимодействий пользователей с фильмами, что позволяет моделям учитывать как предпочтения пользователей, так и характеристики фильмов. Эта матрица является основой для коллаборативной фильтрации, позволяя алгоритмам выявлять скрытые паттерны в данных. Например, если пользователь часто оценивает драмы, система может рекомендовать другие фильмы с аналогичными признаками.

3.2.6 Класс `HyperParameterTuner`

Класс `HyperParameterTuner` предназначен для оптимизации гиперпараметров моделей машинного обучения, что является важным шагом в процессе обучения. Правильная настройка гиперпараметров может значительно повысить производительность модели и улучшить качество рекомендаций. Этот класс предоставляет функции для систематического поиска и оценки оптимальных значений гиперпараметров. Функции приведены ниже:

1 `grid_search(param_grid, X_train, y_train)`. Метод реализует метод перебора (`grid search`), который исследует заданный диапазон значений гиперпараметров для нахождения наилучшей комбинации. Например, если для модели есть гиперпараметры, метод будет оценивать все возможные комбинации этих значений, обучая модель на каждой из них и оценивая результаты. Это позволяет точно настроить параметры для достижения максимальной точности.

2 `random_search(param_dist, X_train, y_train, n_iter)`. Метод реализует случайный поиск (`random search`), который выбирает случайные комбинации гиперпараметров из заданного распределения. Это позволяет исследовать пространство гиперпараметров более эффективно, особенно когда его размер велик. Если есть 10 гиперпараметров, случайный поиск может быстро исследовать различные комбинации, что значительно сокращает время на поиск по сравнению с полным перебором.

3 `bayesian_optimization(param_bounds, X_train, y_train)`. Метод использует байесовскую оптимизацию для нахождения наилучших гиперпараметров. Этот метод строит вероятностную модель функции потерь и использует ее для выбора новых гиперпараметров, которые стоит протестировать. Например, если модель показывает хорошие результаты с определенными значениями гиперпараметров, байесовская оптимизация будет сосредоточена на близлежащих значениях, что делает процесс более целенаправленным и эффективным.

4 `evaluate_params(params, X_train, y_train, X_val, y_val)`. Метод оценивает качество модели на основе заданных гиперпараметров. Он обучает модель с указанными параметрами и использует валидационный набор данных для оценки ее производительности. Например, если модель с определенными гиперпараметрами показывает высокие результаты на валидационном наборе, это может указывать на их эффективность.

5 `best_params()`. Этот метод возвращает наилучшие найденные гиперпараметры после завершения процесса оптимизации. Это позволяет разработчикам легко извлекать результаты и применять их в финальной модели. Если в результате оптимизации было найдено, что количество соседей должно быть 15, а коэффициент регуляризации — 0.1, метод `best_params` предоставит эти значения для дальнейшего использования.

3.3 Блок подготовки исходных данных

Блок подготовки исходных данных ориентирован на извлечение и обработку информации о фильмах из API TMD. Он обеспечивает надежный и эффективный процесс извлечения, обработки и подготовки данных, что критически важно для последующего обучения моделей машинного обучения. Этот блок включает несколько классов, каждый из которых выполняет специфические задачи, направленные на обеспечение качества и актуальности данных.

К основным функциям блока относятся:

- извлечение данных;
- обработка и трансформация данных;
- нормализация данных;
- валидация данных;
- сохранение данных.

За каждую из перечисленных функций отвечает свой класс. Подробнее про классы далее.

3.3.1 Класс `DataLoader`

Класс `TMDbDataLoader` представляет собой ключевой компонент системы, отвечающий за извлечение данных о фильмах из API TMDb. Этот класс обеспечивает надежный и эффективный интерфейс для работы с внешними источниками данных, позволяя поддерживать актуальность и качество информации, используемой в системе рекомендаций.

Основные функции и возможности класса:

1 `init__(self, api_key)`. Инициализация класса осуществляется с использованием API-ключа, что обеспечивает безопасную аутентификацию запросов к API TMDb. В процессе инициализации настраивается базовый URL для API, а также дополнительные параметры взаимодействия, такие как таймауты для запросов. Это позволяет гарантировать стабильность и предсказуемость работы класса в условиях различных сетевых условий.

2 `fetch_movie_data(self, movie_id)`. Данный метод отвечает за извлечение детализированной информации о фильме по его уникальному идентификатору. Он возвращает ключевые атрибуты, такие как название, описание, жанр, актерский состав и рейтинг, что позволяет системе проводить глубокий анализ содержания фильмов. Эта информация служит основой для дальнейшей обработки и формирования рекомендаций, учитывающих предпочтения пользователей.

3 `fetch_popular_movies(self, page=1)`. Метод предназначен для получения списка популярных фильмов, что позволяет системе оперативно обновлять контент и предлагать пользователям наиболее актуальные новинки. Используя данные, предоставляемые API, метод обеспечивает высокую степень актуальности информации и помогает системе адаптироваться к изменяющимся интересам зрителей.

4 `fetch_genres(self)`. Этот метод извлекает список всех доступных жанров фильмов, что является критически важным для классификации контента и улучшения точности рекомендаций. Знание жанров позволяет системе применять контентную фильтрацию, обеспечивая более точное соответствие между предпочтениями пользователей и предлагаемыми фильмами.

5 `safe_fetch(self, url)`. Метод реализует механизм безопасного извлечения данных с обработкой ошибок и исключений. Он обеспечивает устойчивость системы к временным сбоям и другим непредвиденным ситуациям, что позволяет эффективно справляться с проблемами при взаимодействии с API. Это важная функция, которая минимизирует риск сбоев в работе системы и улучшает пользовательский опыт.

6 `cache_data(self, data)`. Реализация механизма кеширования данных, полученных из API, позволяет избежать повторных запросов за одной и той же информацией, что значительно ускоряет процесс извлечения данных и снижает нагрузку на API. Этот подход не только повышает производительность системы, но и обеспечивает экономию ресурсов, что является важным аспектом для масштабируемых приложений.

3.3.2 Класс `MaxFeatureExtractor`

Класс `MovieFeatureExtractor` является неотъемлемой частью блока подготовки данных, обеспечивая извлечение и преобразование ключевых характеристик фильмов, полученных из API TMDb. Его основная задача заключается в том, чтобы преобразовать сырые данные в структурированный и анализируемый формат, что критически важно для системы рекомендаций.

Основные функции и возможности

1 `extract_features(self, movie_data)`. Основной метод класса, отвечающий за извлечение ключевых признаков из полученных данных о фильме. Он обрабатывает информацию, такую как жанры, актерский состав, описание, дата выхода и рейтинги, и преобразует их в структурированный формат, удобный для дальнейшего анализа. Этот метод позволяет выделить

наиболее значимые характеристики, которые могут быть использованы для построения рекомендаций.

2 `transform_genres(self, genres)`. Метод, который преобразует информацию о жанрах в удобный для анализа формат, например, в виде бинарных векторов. Это позволяет системе легко идентифицировать принадлежность фильма к определенным жанрам и использовать эту информацию для контентной фильтрации. Преобразование жанров в числовой формат значительно упрощает процесс обучения моделей машинного обучения.

3 `process_cast(self, cast_data)`. Этот метод отвечает за извлечение и обработку данных о актерском составе фильма. Он формирует список основных актеров и может использовать методы для их кодирования, что позволяет учитывать влияние актеров на предпочтения пользователей. Информация о художниках становится важным фактором в системе рекомендаций, так как зрители часто выбирают фильмы по участию любимых актеров.

4 `normalize_ratings(self, ratings)`. Метод, который нормализует рейтинги фильмов, приводя их к единому стандарту. Это критически важно для обеспечения сопоставимости данных и уменьшения искажений, которые могут возникнуть из-за различий в диапазонах оценок. Нормализация позволяет повысить точность анализа и улучшить качество рекомендаций.

5 `extract_keywords(self, keywords)`. Метод для извлечения и обработки ключевых слов, связанных с фильмом. Ключевые слова могут использоваться для улучшения контентной фильтрации и повышения точности рекомендаций, так как они помогают более точно описывать содержание фильма. Этот метод может включать в себя алгоритмы для определения наиболее значимых и частых ключевых слов.

3.3.3 Класс `DataNormalizer`

Класс `DataNormalizer` является важным компонентом системы подготовки данных, отвечающим за нормализацию и стандартизацию признаков, извлеченных из фильмов. Его основная задача заключается в обеспечении согласованности и сопоставимости данных, что критически важно для повышения качества анализа и точности моделей машинного обучения. Нормализация данных позволяет избежать искажений и улучшает интерпретацию результатов.

Основные функции и возможности:

1 `normalize_ratings(self, ratings)`. Метод, ответственный за нормализацию рейтингов фильмов. Он приводит рейтинги к единому стандарту, что позволяет устранить различия в диапазонах оценок, которые могут возникнуть из-за особенностей различных платформ. Нормализация рейтингов способствует повышению сопоставимости данных и улучшает результаты анализа.

2 `standardize_features(self, features)`. Этот метод осуществляет

стандартизацию признаков, приводя их к нормальному распределению с нулевым средним и единичной дисперсией. Стандартизация позволяет обеспечить равный вес всех признаков в процессе обучения моделей, что особенно важно для алгоритмов, чувствительных к масштабу данных, таких как SVM или KNN.

3 `min_max_scaling(self, data)`. Метод, реализующий масштабирование признаков в определенный диапазон, обычно от 0 до 1. Это позволяет убедиться, что все признаки находятся в одинаковом масштабе, что критически важно для алгоритмов, использующих метрики расстояния. Масштабирование улучшает сходимость и стабильность алгоритмов обучения.

4 `handle_missing_values(self, data)`. Метод, отвечающий за обработку пропущенных значений в данных. Он может включать в себя различные стратегии, такие как заполнение средними значениями, медианами или использованием методов интерполяции. Обработка пропусков важна для обеспечения целостности данных и предотвращения искажений в процессе анализа.

5 `normalize_text(self, text_data)`. Этот метод отвечает за нормализацию текстовых данных, таких как описания фильмов. Он может включать в себя преобразование текста в нижний регистр, удаление стоп-слов и лемматизацию, что позволяет улучшить качество анализа текстовой информации. Нормализация текста помогает выделить ключевые слова и улучшает качество рекомендаций.

3.3.4 Класс `MovieDataValidator`

Класс `MovieDataValidator` является важным компонентом системы, отвечающим за проверку и валидацию данных о фильмах, получаемых из API TMDb. Его основная задача заключается в обеспечении целостности и корректности данных, что критически важно для дальнейшего анализа и обработки. Этот класс помогает выявлять и устранять ошибки на ранних этапах, что минимизирует риск неправильного представления информации пользователям.

Основные функции и возможности класса:

1 `validate_movie_data(self, movie_data)`. Основной метод, ответственный за валидацию данных о фильме. Он проверяет наличие обязательных полей, таких как название, описание и рейтинг, а также их соответствие ожидаемым форматам и диапазонам. Этот метод помогает исключить неполные или некорректные записи, что повышает общее качество данных.

2 `check_rating(self, rating)`. Метод, который осуществляет проверку корректности значений рейтинга. Он гарантирует, что рейтинги находятся в допустимом диапазоне (например, от 0 до 10) и соответствуют ожидаемым форматам. Валидация рейтингов важна для обеспечения достоверности информации, представляемой пользователям.

3 `validate_genres(self, genres)`. Этот метод отвечает за проверку корректности списка жанров, связанных с фильмом. Он проверяет, что жанры существуют в предопределенном наборе допустимых значений и соответствуют формату. Эта функция позволяет избежать ошибок при классификации фильмов и улучшает точность рекомендаций.

4 `check_release_date(self, release_date)`. Метод, который осуществляет проверку правильности формата даты выхода фильма. Он гарантирует, что дата соответствует стандартному формату и находится в разумных пределах (например, не позже текущего года). Валидация даты выхода важна для представления актуальной информации о фильмах.

5 `handle_invalid_data(self, invalid_data)`. Метод, отвечающий за обработку некорректных данных, выявленных в ходе валидации. Он может включать логику для журналирования ошибок, уведомления разработчиков или автоматической коррекции, если это возможно. Эффективная обработка некорректных данных позволяет минимизировать влияние ошибок на систему и улучшает общее качество данных.

3.3.5 Класс `MovieDataSaver`

Класс `MovieDataSaver` является важным компонентом системы, отвечающим за сохранение и управление данными о фильмах после их извлечения и обработки. Этот класс обеспечивает надежное и эффективное хранение данных, что критически важно для дальнейшего анализа, использования и предоставления пользователям. Он поддерживает различные форматы хранения и методы для обеспечения целостности данных.

Основные функции и возможности:

1 `init__(self, storage_path)`. Инициализация класса с указанием пути к хранилищу данных. Этот параметр позволяет настроить место, где будут сохраняться файлы, обеспечивая гибкость и удобство в управлении данными. Инициализация может также включать в себя настройки для подключения к базам данных, если используется более сложная архитектура хранения.

2 `save_to_json(self, movie_data)`. Метод, отвечающий за сохранение данных о фильме в формате JSON. Он преобразует структурированные данные в JSON-формат и записывает их в указанный файл. Этот метод обеспечивает легкость в обмене данными и их переносимости, что полезно для интеграции с другими системами.

3 `save_to_csv(self, movie_data)`. Метод для сохранения данных в формате CSV. Он преобразует информацию о фильмах в табличный формат, что обеспечивает удобство работы с данными в аналитических инструментах и электронных таблицах. CSV-формат также позволяет легко импортировать данные в другие системы или базы данных.

4 `save_to_database(self, movie_data, db_connection)`. Метод, который отвечает за сохранение данных в реляционной базе данных. Он использует переданное соединение с базой данных для выполнения операций

вставки и обновления. Этот подход обеспечивает возможность структурированного хранения и быстрого доступа к данным, что критически важно для масштабируемых приложений.

5 `handle_existing_data(self, movie_id, movie_data)`. Метод для обработки ситуации, когда данные о фильме уже существуют в хранилище. Он может включать логику для обновления существующих записей, добавления новых данных или их удаления в зависимости от требований системы. Эффективное управление существующими данными позволяет поддерживать актуальность и целостность информации.

6 `log_save_operation(self, operation_details)`. Метод, отвечающий за ведение журнала операций сохранения данных. Это может включать запись информации о времени операции, статусе и возможных ошибках. Логирование операций помогает в диагностике и мониторинге системы, позволяя отслеживать изменения и обеспечивать прозрачность процессов.

3.3.6 Класс `DataTransformer`

Класс `DataTransformer` является важным компонентом системы, отвечающим за преобразование и предварительную обработку данных о фильмах. Его основная задача заключается в преобразовании сырых данных в удобный для анализа и обработки формат. Этот класс обеспечивает подготовку данных для последующего анализа, обучения моделей и формирования рекомендаций.

Основные функции и возможности:

1 `transform_movie_data(self, raw_data)`. Основной метод класса, отвечающий за преобразование сырых данных о фильмах. Он принимает необработанные данные и выполняет их обработку, включая нормализацию, стандартизацию и кодирование признаков. Этот метод позволяет привести данные к единому формату, что критически важно для последующего анализа и использования.

2 `encode_categorical_features(self, data)`. Метод, который отвечает за кодирование категориальных признаков, таких как жанры и актерский состав. Он использует различные техники, включая one-hot encoding и label encoding, чтобы преобразовать текстовые данные в числовой формат. Это позволяет алгоритмам машинного обучения эффективно обрабатывать категориальные переменные.

3 `scale_numeric_features(self, data)`. Метод, осуществляющий масштабирование числовых признаков. Он позволяет привести данные к единому масштабу, например, с использованием Min-Max Scaling или Z-score нормализации. Масштабирование критически важно для алгоритмов, чувствительных к величине признаков, таких как KNN или линейная регрессия.

4 `transform_text_data(self, text_data)`. Этот метод отвечает за предварительную обработку текстовых данных, таких как описания фильмов.

Он может включать такие процедуры, как токенизация, удаление стоп-слов и стемминг, что позволяет улучшить качество анализа текстовой информации и выделение ключевых слов.

`5 combine_features(self, features_list).` Метод, который объединяет различные признаки в единый набор данных. Он может использоваться для создания новых признаков на основе существующих, что позволяет улучшить представление данных и повысить эффективность анализа. Комбинирование признаков помогает выявить скрытые зависимости и улучшить качество рекомендаций.

3.4 Блок клиентского приложения

Блок клиентского приложения представляет собой фронтенд-часть системы, которая обеспечивает взаимодействие пользователей с функциональностью, реализованной на бэкенде. В данном случае, фронтенд будет разрабатываться с использованием Vue.js, что позволит создать отзывчивый и интерактивный интерфейс. Этот блок отвечает за отображение данных, обработку пользовательских запросов и взаимодействие с API.

К основным функциям и компонентам блока относятся:

- интерфейс пользователя;
- маршрутизация;
- управление состоянием;
- взаимодействие в сервером;
- поиск и фильтрация;
- адаптивный дизайн.

Интерфейс пользователя (UI) является одним из основных аспектов этого блока. Он состоит из различных компонентов Vue, которые отображают информацию о фильмах, такие как карточки с описанием, изображения, рейтинги и другие ключевые атрибуты. Каждый компонент разрабатывается с учетом принципа переиспользования, что позволяет легко изменять и настраивать интерфейс в зависимости от потребностей пользователей. Стилизация интерфейса осуществляется с использованием CSS-фреймворков, таких как Bootstrap или Tailwind CSS, что помогает создать современный и привлекательный дизайн. Это делает приложение более интуитивным и удобным для пользователей, позволяя им легко ориентироваться в доступном контенте.

Маршрутизация в приложении организована с помощью Vue Router. Это позволяет пользователю перемещаться между различными страницами, такими как главная страница, страница деталей фильма и страница поиска. Динамические маршруты обеспечивают возможность отображения информации о конкретных фильмах на основе их идентификаторов, что делает навигацию более удобной и логичной. Пользователи могут быстро получать доступ к интересующим их данным, что улучшает общий опыт работы с приложением.

Для управления состоянием приложения используется Vuex, что

позволяет централизовать данные о фильмах, предпочтениях пользователей и других аспектах.

Взаимодействие с сервером осуществляется с помощью библиотеки *Axios*, которая позволяет выполнять HTTP-запросы к бэкенду, получая данные о фильмах, жанрах и рекомендациях. Обработка ответов от API включает в себя управление возможными ошибками и отображение пользователям соответствующих уведомлений. Это обеспечивает плавный и бесперебойный процесс получения данных, позволяя пользователям получать актуальную информацию без задержек.

Функционал поиска и фильтрации также играет важную роль в приложении. Пользователи могут легко находить фильмы по названию, жанру или актеру, что значительно улучшает их опыт. Результаты поиска отображаются в реальном времени, что делает процесс поиска более интерактивным. Возможность фильтрации фильмов по различным критериям, таким как рейтинг или дата выхода, позволяет пользователям быстро находить нужный контент, учитывая их предпочтения и интересы.

Адаптивный дизайн приложения обеспечивает его корректное отображение на различных устройствах, включая мобильные телефоны и планшеты. Использование медиазапросов и гибкой верстки гарантирует, что интерфейс будет выглядеть привлекательно и оставаться функциональным на экранах разных размеров. Тестирование на различных устройствах и браузерах позволяет выявить и устранить возможные проблемы с совместимостью, что дополнительно улучшает пользовательский опыт.

В целом, блок клиентского приложения, реализованный на *Vue.js*, обеспечивает надежное и эффективное взаимодействие пользователей с системой. Он сочетает в себе интуитивный интерфейс, мощные функции поиска и фильтрации, а также надежное управление состоянием и взаимодействие с API, что делает его ключевым элементом успешного проекта.

3.7 Блок реляционной базы данных

Блок реляционной базы данных играет критическую роль в структуре системы, обеспечивая надежное хранение и управление данными о фильмах, пользователях и других связанных сущностях. Этот блок отвечает за организацию данных, их целостность и доступность, что позволяет эффективно извлекать и обрабатывать информацию для дальнейшего анализа и использования в приложении.

Для взаимодействия с реляционной базой данных на стороне бэкенда будут реализованы несколько классов, каждый из которых отвечает за различные аспекты работы с данными.

`1 MovieRepository`. Этот класс будет отвечать за операции с таблицей фильмов. Он реализует методы для создания, чтения, обновления и удаления (CRUD) записей о фильмах. Также класс будет включать методы для получения популярных фильмов, фильмов по жанрам и актерскому составу,

что обеспечит гибкость в извлечении данных.

2 `UserRepository`. Класс, отвечающий за управление данными пользователей. Он будет включать методы для регистрации пользователей, аутентификации, обновления информации о пользователях и получения их предпочтений. Этот класс обеспечит безопасность и целостность данных пользователей.

3 `GenreRepository`. Этот класс будет управлять таблицей жанров, обеспечивая операции CRUD для жанров фильмов. Он также будет включать методы для получения всех доступных жанров и их ассоциации с фильмами.

4 `RatingRepository`. Класс, отвечающий за управление данными о рейтингах и оценках фильмов. Он будет включать методы для добавления, обновления и извлечения оценок, что позволит пользователям оставлять отзывы и оценки на фильмы.

5 `KeywordRepository`. Этот класс будет заниматься управлением ключевыми словами, связанными с фильмами. Он будет обеспечивать функции для добавления, удаления и извлечения ключевых слов, что поможет улучшить поиск и фильтрацию фильмов.

6 `RecommendationService`. Класс, который будет реализовывать логику рекомендаций на основе собранных данных. Он будет использовать информацию из различных репозиторий и алгоритмы для формирования персонализированных рекомендаций для пользователей.

В реляционной базе данных будут созданы несколько таблиц, каждая из которых будет содержать информацию о различных сущностях системы. Таблицы и их поля описаны ниже:

1 `movies`. Эта таблица будет хранить информацию о фильмах. Она будет включать следующие поля:

- `id` (PRIMARY KEY, INT) — уникальный идентификатор фильма.
- `title` (VARCHAR) — название фильма.
- `description` (TEXT) — описание фильма.
- `release_date` (DATE) — дата выхода.
- `rating` (FLOAT) — средний рейтинг.
- `created_at` (TIMESTAMP) — дата и время создания записи.
- `updated_at` (TIMESTAMP) — дата и время последнего обновления записи.

2 `users`. Таблица, хранящая информацию о пользователях системы. Поля включают:

- `id` (PRIMARY KEY, INT) — уникальный идентификатор пользователя.
- `username` (VARCHAR) — имя пользователя.
- `password_hash` (VARCHAR) — хэш пароля.
- `email` (VARCHAR) — адрес электронной почты.
- `created_at` (TIMESTAMP) — дата и время создания

пользователя.

- `updated_at` (TIMESTAMP) — дата и время последнего обновления информации о пользователе.

3 `genres`. Таблица, содержащая все доступные жанры. Поля:

- `id` (PRIMARY KEY, INT) — уникальный идентификатор жанра.
- `name` (VARCHAR) — название жанра.

4 `movie_genre`. Ассоциативная таблица для связывания фильмов и жанров. Поля:

- `movie_id` (FOREIGN KEY, INT) — идентификатор фильма.
- `genre_id` (FOREIGN KEY, INT) — идентификатор жанра.

5 `ratings`. Таблица, хранящая оценки пользователей для фильмов. Поля:

- `id` (PRIMARY KEY, INT) — уникальный идентификатор оценки.
- `user_id` (FOREIGN KEY, INT) — идентификатор пользователя.
- `movie_id` (FOREIGN KEY, INT) — идентификатор фильма.
- `rating` (FLOAT) — оценка, оставленная пользователем.

6 `keywords`. Таблица для хранения ключевых слов, связанных с фильмами. Поля:

- `id` (PRIMARY KEY, INT) — уникальный идентификатор ключевого слова.
- `word` (VARCHAR) — само ключевое слово.

7 `movie_keyword`. Ассоциативная таблица для связывания фильмов и ключевых слов. Поля:

- `movie_id` (FOREIGN KEY, INT) — идентификатор фильма.
- `keyword_id` (FOREIGN KEY, INT) — идентификатор ключевого слова.

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

В данном разделе представлено несколько ключевых алгоритмов программного средства, а также приведено описание нескольких ключевых представлений, реализованных в рамках системы рекомендаций фильмов с использованием искусственного интеллекта.

4.1 Общая архитектура системы

Программное средство реализовано с использованием языка программирования Python. В качестве основного инструмента для построения рекомендательной системы использована библиотека `scikit-learn`, а для анализа данных — `pandas` и `numpy`. Интерфейс пользователя будет реализован на фреймворке `Vue.js`, что обеспечит современный и интерактивный веб-интерфейс для взаимодействия пользователя с рекомендательной системой. На текущем этапе фокус сделан на серверной части и логике формирования рекомендаций.

Система строит рекомендации на основе контентного фильтрационного подхода, при котором каждый фильм описывается совокупностью признаков: жанр, режиссер, актерский состав, ключевые слова, описание сюжета и др. Эти признаки используются для построения векторного представления каждого фильма.

4.2 Ключевые алгоритмы и функции

Для описания выбрано только несколько представлений, при этом упор сделан на представления, которые используются как шаблоны или части включаются в другие представления.

4.2.1 Предобработка данных

Предобработка включает в себя очистку, нормализацию и агрегирование информации о фильмах. Данные поступают из двух различных источников: основной таблицы с фильмами и дополнительной таблицы с информацией о съемочной группе. Эти таблицы объединяются по названию фильма, после чего из полученного датафрейма отбираются только необходимые столбцы.

На этом этапе осуществляется удаление строк с пропущенными значениями, что особенно важно для корректного обучения моделей. Также проводится структурное преобразование некоторых полей: JSON-подобные строки (в жанрах, актерах, ключевых словах и т.д.) парсятся с помощью `ast.literal_eval()` и преобразуются в списки. В некоторых случаях отбираются только первые три значения (например, три главных актера), что снижает размерность данных и повышает релевантность признаков.

В результате предобработки создается поле `tags`, которое объединяет в себе все важные текстовые характеристики фильма. Это поле формируется

путем конкатенации жанров, описания, ключевых слов, состава актеров и режиссера. Объединенное текстовое описание фильма становится основой для последующей векторизации.

```
def preprocess_data(movies_df, credits_df):
    movies = movies_df.merge(credits_df, on='title')
    movies = movies[['movie_id', 'title', 'overview',
'genres', 'keywords', 'cast', 'crew']]

    movies.dropna(inplace=True)

    def convert(obj):
        L = []
        for i in ast.literal_eval(obj):
            L.append(i['name'])
        return L

    movies['genres'] = movies['genres'].apply(convert)
    movies['keywords'] = movies['keywords'].apply(convert)
    movies['cast'] = movies['cast'].apply(lambda x:
[i['name'] for i in ast.literal_eval(x)[:3]])
    movies['crew'] = movies['crew'].apply(lambda x:
[i['name'] for i in ast.literal_eval(x) if i['job'] ==
'Director'])

    movies['tags'] = movies['overview'] + ' ' +
movies['genres'].apply(lambda x: ' '.join(x)) + ' ' + \
    movies['keywords'].apply(lambda x: '
'.join(x)) + ' ' + \
    movies['cast'].apply(lambda x: '
'.join(x)) + ' ' + \
    movies['crew'].apply(lambda x: '
'.join(x))

    return movies[['movie_id', 'title', 'tags']]
```

Алгоритм предобработки данных следующий:

1. Объединение таблиц. Объединяются два датафрейма `movies_df` и `credits_df` по столбцу `title`.
2. Выбор нужных столбцов. Из объединённого датафрейма отбираются только нужные поля: `movie_id`, `title`, `overview`, `genres`, `keywords`, `cast`, `crew`.
3. Удаление пропущенных значений. Удаляются все строки, содержащие хотя бы одно `NaN`.
4. Парсинг строк с JSON-структурой
 - `genres` и `keywords` преобразуются из строк в списки названий;
 - `cast` – из строки выбираются имена трех первых актёров;
 - `crew` – извлекается имя режиссёра из состава съёмочной группы.
5. Создание текстового поля `tags`. Формируется единое поле `tags`,

содержащее текстовое описание фильма: `overview` + жанры + ключевые слова + актёры + режиссёр (всё объединяется в одну строку).

6. Возврат итогового датафрейма. Возвращается датафрейм с тремя колонками: `movie_id`, `title`, `tags` — где `tags` готов для векторизации.

4.2.2 Векторизация текста и вычисление схожести

После этапа предобработки каждый фильм описывается текстовым полем `tags`, которое содержит сводную информацию о жанре, актерах, ключевых словах, описании и режиссере. Однако для того, чтобы система могла анализировать и сравнивать фильмы между собой, необходимо перевести этот текст в числовой формат. Для этой цели применяется метод векторизации текста — перевод строк в векторы фиксированной размерности.

Одним из наиболее эффективных инструментов для этой задачи является класс `CountVectorizer` из библиотеки `scikit-learn`. Он преобразует текст в так называемую «мешковую модель слов» (`bag-of-words`), при которой каждое уникальное слово становится отдельной координатой в векторном пространстве. Частота слов в документе фиксируется, что позволяет сохранить значимую информацию о его содержании.

Для сокращения размерности пространства устанавливается ограничение `max_features=5000`, а также исключаются часто встречающиеся, но малозначимые слова с помощью параметра `stop_words='english'`.

После преобразования всех фильмов в векторное пространство, применяется функция `cosine_similarity`, которая рассчитывает косинусное сходство между векторами. Косинусное сходство варьируется от 0 до 1 и позволяет определить степень схожести между двумя фильмами: чем ближе значение к 1, тем более похожи фильмы друг на друга по содержанию.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

cv = CountVectorizer(max_features=5000,
stop_words='english')
vectors = cv.fit_transform(movies['tags']).toarray()

similarity = cosine_similarity(vectors)
```

Алгоритм векторизации и расчета сходства по шагам:

1. Инициализация `CountVectorizer` с ограничением на количество признаков и стоп-словами.
2. Преобразование текстов из поля `tags` в числовые векторы.
3. Преобразование полученной разреженной матрицы в обычный массив.
4. Вычисление косинусного сходства между каждым парой фильмов.
5. Получение матрицы `similarity`, в которой каждая ячейка `[i][j]` отражает степень близости фильмов `i` и `j`.

4.2.3 Формирование рекомендаций

На основе полученной матрицы косинусных расстояний система строит рекомендации для пользователя. Функция `recommend()` принимает название фильма и возвращает список наиболее похожих фильмов. В основе этого процесса лежит идея, что у каждого фильма есть уникальный вектор признаков, и фильмы, схожие по этим признакам, будут иметь наименьшее расстояние в векторном пространстве.

```
def recommend(movie, df, similarity):
    movie_index = df[df['title'] == movie].index[0]
    distances = similarity[movie_index]
    movie_list = sorted(list(enumerate(distances)),
reverse=True, key=lambda x: x[1])[1:6]
    recommendations = []
    for i in movie_list:
        recommendations.append(df.iloc[i[0]].title)
    return recommendations
```

Алгоритм формирования рекомендаций работает следующим образом:

1. Получается индекс фильма, выбранного пользователем.
2. Из матрицы `similarity` извлекаются все расстояния до остальных фильмов.
3. Эти расстояния сортируются по убыванию схожести (по убыванию значения косинусного сходства).
4. Из отсортированного списка выбираются топ-5 наиболее похожих фильмов, исключая сам фильм.
5. Формируется список названий рекомендованных фильмов.

4.2.4 Пользовательский интерфейс

Взаимодействие с рекомендательной системой осуществляется через вызов функции `recommend(movie)`, возвращающей список фильмов, схожих с выбранным пользователем. Однако с учетом будущего развертывания на веб-платформе, пользовательский интерфейс будет реализован с применением фреймворка `Vue.js` и обеспечит полнофункциональное и интуитивно понятное взаимодействие с системой.

Интерфейс будет включать следующие ключевые компоненты:

1. Поле поиска фильма позволяет пользователю ввести название интересующего фильма. После ввода появляется выпадающий список совпадающих фильмов, упрощающий выбор.
2. Кнопка получения рекомендаций активирует отправку запроса к серверу для получения рекомендаций на основе выбранного фильма.
3. Список рекомендованных фильмов отображает полученные от сервера фильмы с краткой информацией (постер, название, жанр и рейтинг). Каждый элемент списка можно кликнуть для получения расширенной информации о фильме.

4. Карточка информации о фильме при выборе фильма отображает подробную информацию, включая описание, жанры, актерский состав и трейлер (если доступен).

5. Адаптивная навигация обеспечивает возможность перемещения между разделами (например, "Главная", "Избранное", "Настройки").

Обмен данными между интерфейсом и серверной частью будет осуществляться через REST API. Для отправки и получения запросов будет использоваться библиотека `axios`. Пример типичного запроса:

```
axios.post('/api/recommend', {
  title: selectedMovieTitle
})
.then(response => {
  this.recommendations = response.data.recommendations;
})
.catch(error => {
  console.error('Ошибка получения рекомендаций:', error);
});
```

Результаты передаются в формате JSON. Компоненты Vue будут реактивно обновлять отображаемую информацию на основе данных, полученных от сервера.

5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

В данном разделе представлена методика испытания программного средства. Проведено некоторое тестирование функционала и графического интерфейса.

5.1 Тесты графического интерфейса

Для проверки графического интерфейса веб-приложения использовались инструменты разработчика браузера, позволяющие оценить поведение интерфейса при различных условиях, таких как:

- изменение масштаба интерфейса и плотности пикселей
- ориентация экрана (портретная и ландшафтная);
- различные разрешения экрана (для проверки адаптивности интерфейса на мобильных устройствах, планшетах и десктопах).

Тестирование проводилось в адаптивном режиме браузера, что позволило убедиться в корректном отображении элементов интерфейса на экранах с различными размерами и пропорциями.

5.1.1 Адаптивность интерфейса для разных устройств

Для проверки адаптивности интерфейса было проведено его тестирование на устройствах с разными размерами дисплея. На рисунке 5.1 изображен интерфейс на смартфоне с расширением экрана 425 на 659 пикселей, а на рисунке 5.2 пример тестирования на ноутбуке с расширением экрана 1920 на 1080 пикселе:

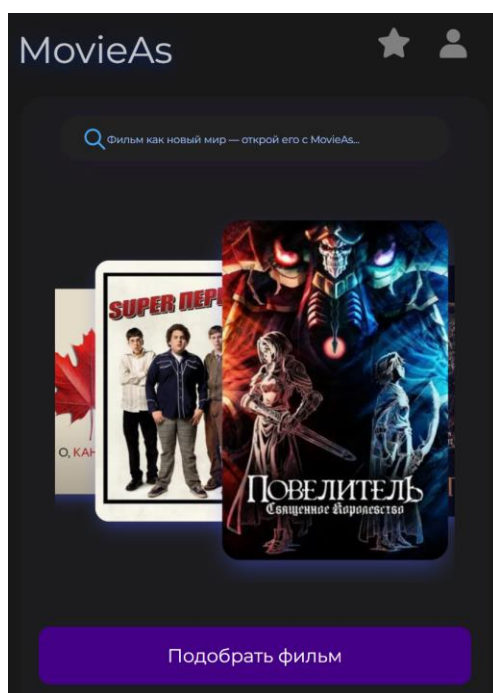


Рисунок 5.1 – Главная страница на смартфоне

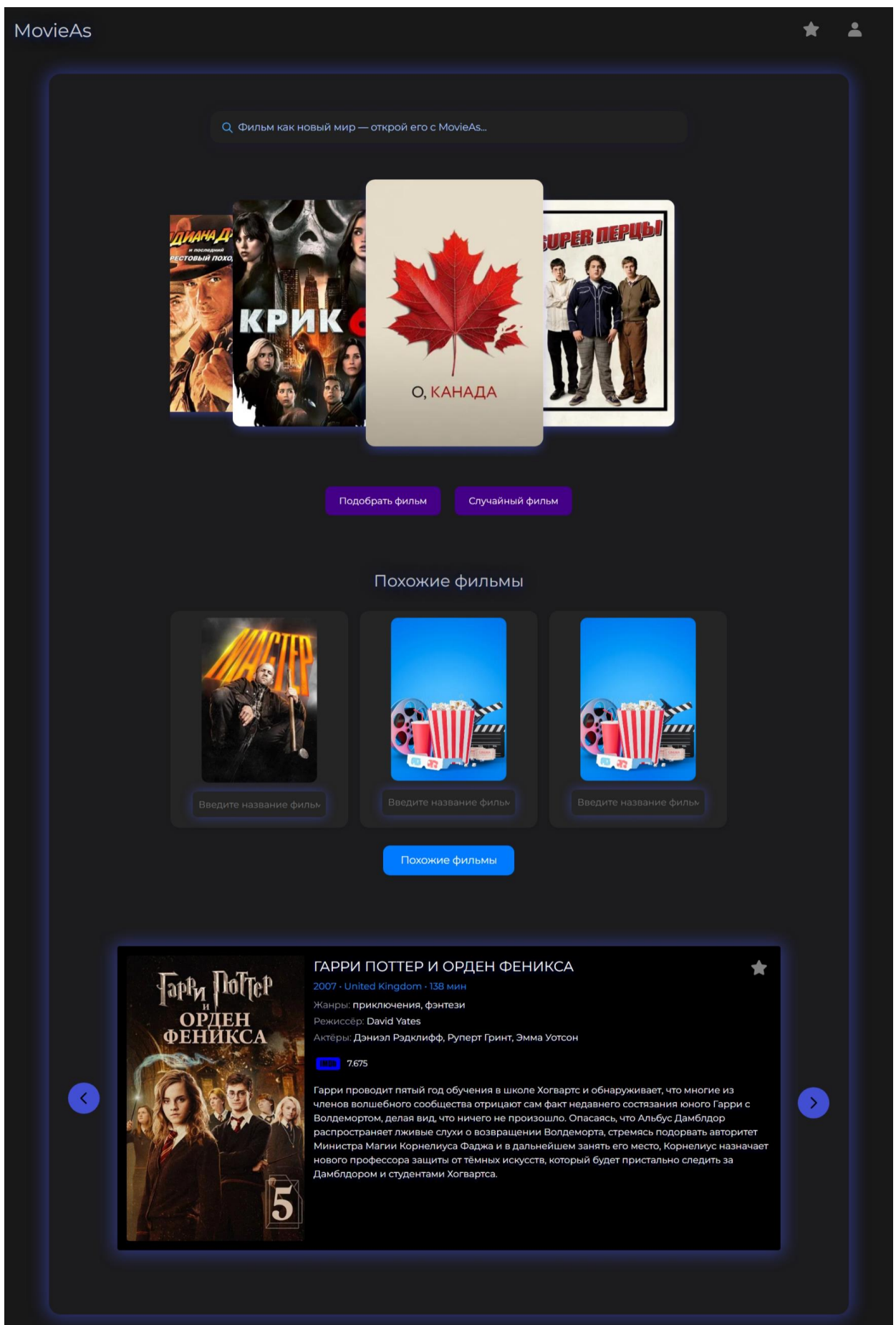


Рисунок 5.2 – Главная страница на ноутбуке

5.1.2 Масштаб интерфейса

В современных браузерах предусмотрено системное масштабирование интерфейса, которое позволяет изменять размер текста и других элементов в соответствии с настройками пользователя.

Для тестирования использовались следующие уровни масштабирования интерфейса, условно разбитые на группы:

- уменьшенные масштабы (например, 80%, 90%);
- стандартный масштаб (100%);
- увеличенные масштабы (125%, 150%);
- повышенные значения масштабирования (175%, 200%).

Так как значения масштабирования выше 150% встречаются сравнительно редко и могут потребовать дополнительной адаптации макета, в рамках тестирования рассматривались только первые три уровня.

Согласно результатам проверки, все компоненты интерфейса корректно масштабируются: текст остаётся читаемым, элементы не выходят за пределы своих контейнеров, и структура макета сохраняется.

Результаты теста представлены на рисунке:

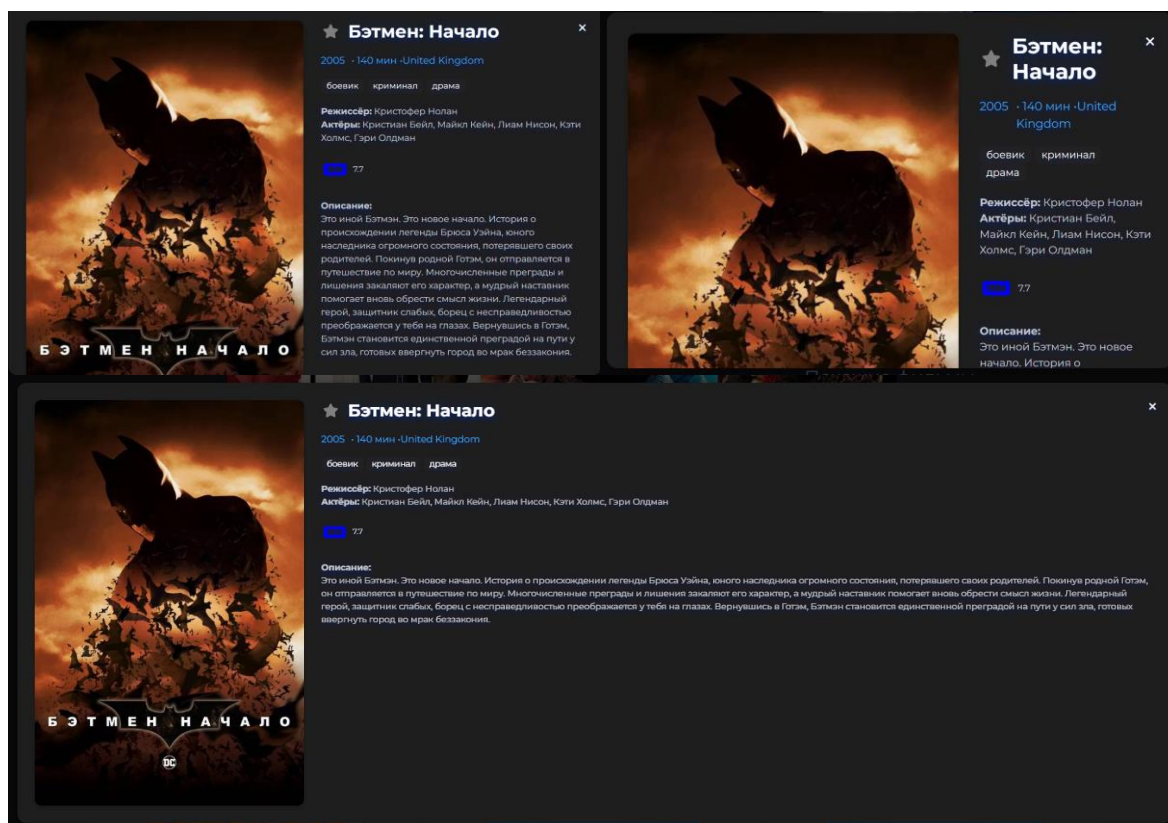


Рисунок 5.3 – Представление детальной информации о фильме с различными масштабами интерфейса

5.1.3 Ориентация интерфейса

Интерфейс приложения должен адаптироваться к разным ориентациям

экрана. Например, при переходе в горизонтальную (ландшафтную) ориентацию в одну строку должно помещаться больше элементов интерфейса.

Для проверки этого поведения использовались инструменты разработчика браузера, позволяющие вручную изменять размеры и пропорции экрана. Тестирование всех ключевых представлений проводилось как в портретной, так и в ландшафтной ориентации.

Результаты тестирования представлены на рисунке: интерфейс корректно перестраивается в зависимости от ориентации, обеспечивая удобство использования на различных устройствах.

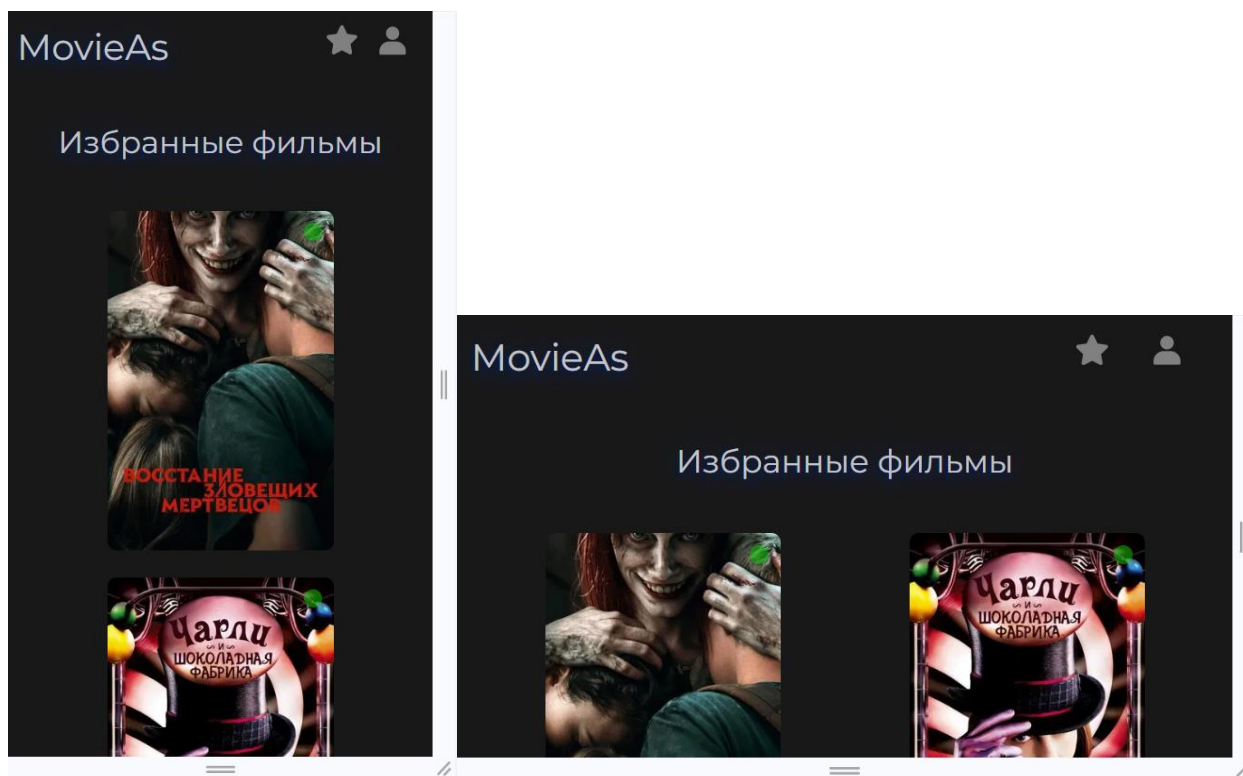


Рисунок 5.4 – Представление избранного при различных ориентациях

5.2 Тесты функциональности

Тестирование функциональности проводится с целью выявления возможных ошибок в реализации основных возможностей веб-приложения, которые могут привести к некорректной обработке данных или сбоям в работе интерфейса.

Основное внимание уделяется следующим функциям:

- поиск фильмов по названию;
- подбор рекомендаций на основе выбранных фильмов;
- отображение похожих фильмов;
- добавление и удаление фильмов из избранного.

Поскольку приложение использует локальную базу данных, часть функционала может работать и без сетевого соединения, однако доступ к внешнему API (например, для получения обновлённой информации о

фильмах) возможен только при наличии подключения к интернету.

Результаты тестирования приведены в таблице:

Таблица 5.1 – Результаты тестирования без сетевого соединения

Тест	Ожидаемый результат	Полученный результат
Вход в приложение	Отображается главная страница	Отображается главная страница
Поиск фильма	Отображается список в соответствии с поисковым запросом	Отображается список в соответствии с поисковым запросом
Просмотр детальной информации о фильме	Отображается карточка с информацией	Отображается карточка с информацией
Подбор фильма	Отображается список фильмов по критериям	Отображается список фильмов по критериям
Случайный фильм	Отображается карточка со случайным фильмом	Отображается карточка со случайным фильмом
Похожие фильмы	Отображается 10 карточек с похожими фильмами	Отображается 10 карточек с похожими фильмами
Просмотр избранных	Отображается страница с избранными фильмами	Отображается страница с избранными фильмами

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

6.1 Системные требования

Для работы веб-приложения необходимо любое устройство с современным веб-браузером и подключением к сети интернет. Приложение корректно функционирует в последних версиях следующих браузеров:

- Google Chrome (рекомендуется версия 100 и выше);
- Mozilla Firefox (версия 100 и выше);
- Safari (на устройствах с iOS, iPadOS и macOS — версия 15 и выше);
- Microsoft Edge (версия 100 и выше).

Приложение адаптировано под работу на следующих типах устройств:

- настольные компьютеры и ноутбуки (Windows, macOS, Linux);
- планшеты (iPad, Android-планшеты);
- смартфоны (iPhone, Android).

Для полноценного функционирования, включая поиск фильмов, отображение рекомендаций и добавление в избранное, требуется активное подключение к интернету, так как данные загружаются с внешнего API. Без доступа к сети отдельные функции могут быть недоступны.

6.2 Доступ к приложению

После публикации приложения ссылка на его страницу будет размещена в репозитории проекта на GitHub [9].

Для запуска веб-приложения достаточно перейти по указанной ссылке в любом современном браузере — установка не требуется. При первом запуске потребуется активное подключение к интернету для загрузки всех необходимых данных.

6.3 Краткое руководство пользователя

При первом запуске веб-приложения пользователю отображается главная страница. Незарегистрированный пользователь имеет доступ к базовому функционалу, такому как просмотр информации о фильмах и поиск по каталогу.

После авторизации доступный функционал значительно расширяется: становится возможным добавление фильмов в избранное, получение персонализированных рекомендаций.

Полный перечень функциональных возможностей представлен в разделе ниже.

6.3.1 Подбор фильма

На главной странице веб-приложения размещена кнопка «Подобрать фильм», предназначенная для персонализированного поиска кино по

заданным критериям. После нажатия на эту кнопку открывается модальное окно, в котором реализован пошаговый алгоритм выбора параметров. Процесс подбора фильма состоит из следующих этапов:

1. Выбор жанра. На первом этапе пользователь выбирает интересующий его жанр фильма из предложенного списка. Это может быть, например, комедия, драма, фантастика, триллер и др. Жанровая фильтрация помогает сузить круг поиска в соответствии с предпочтениями пользователя.

2. Указание года или диапазона годов выпуска. На следующем этапе предлагается указать либо конкретный год выпуска фильма, либо задать диапазон лет (например, с 2010 по 2020 год). Это позволяет учитывать как современные, так и более старые фильмы, в зависимости от интереса пользователя.

3. Выбор страны производства. Третий шаг включает выбор страны производства. Данный критерий особенно полезен, если пользователь предпочитает фильмы конкретных кинематографий (например, американские, французские, корейские и т. д.).

После завершения всех трёх этапов и выбора параметров пользователь нажимает кнопку «Подтвердить». В результате происходит генерация результатов подбора — на экран выводится список фильмов, соответствующих указанным критериям.

Список фильмов (рисунок 6.1) представлен в виде галереи карточек, каждая из которых содержит основную информацию о фильме: название, постер, год выпуска. Такой формат позволяет быстро просматривать и сравнивать предложенные варианты.

При клике на одну из карточек открывается страница с подробной информацией о фильме. В этом окне пользователь может ознакомиться с более полной информацией, включая описание сюжета, список актёров, рейтинги, трейлер (при наличии), а также добавить фильм в избранное, чтобы вернуться к нему позже.

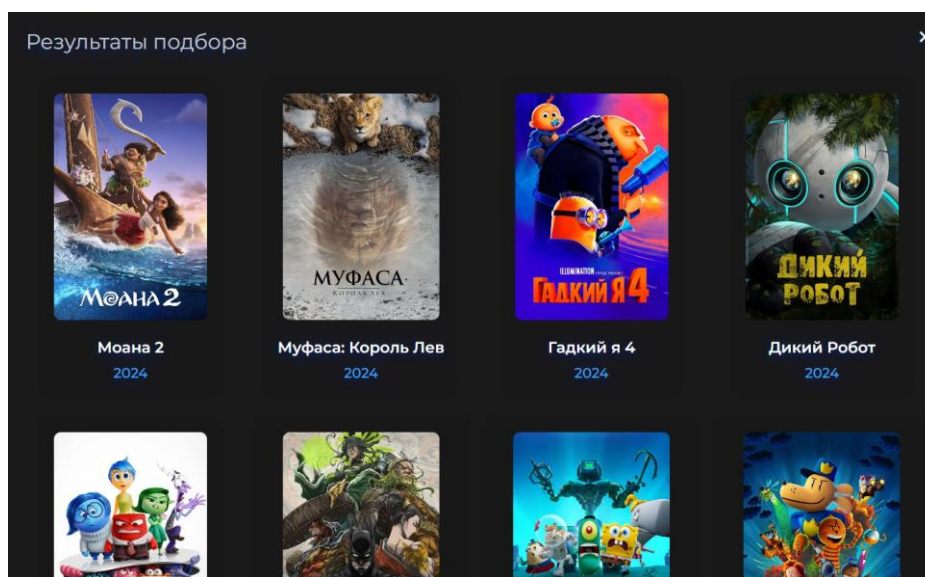


Рисунок 6.1 – Список подобранных фильмов

6.3.2 Похожие фильмы

На рисунке 6.2 представлена секция приложения, предназначенная для подбора похожих фильмов. Данная функция позволяет пользователю получить индивидуальные рекомендации на основе ранее просмотренных или понравившихся фильмов.

Пользователь может ввести от одного до трёх названий фильмов, которые ему интересны. После подтверждения выбора запускается алгоритм анализа, основанный на различных критериях — таких как жанр, стиль повествования, атмосфера, актёрский состав, рейтинг и другие характеристики.

На основе этого анализа формируется список рекомендуемых фильмов, обладающих сходными признаками. Результаты подбора отображаются в виде карточек фильмов с подробной информацией по каждому из них (рисунок 6.3). Каждая карточка включает в себя название фильма, обложку, год выпуска, продолжительность, страну производства, актеров, режиссеров, жанры, рейтинг и краткое описание.

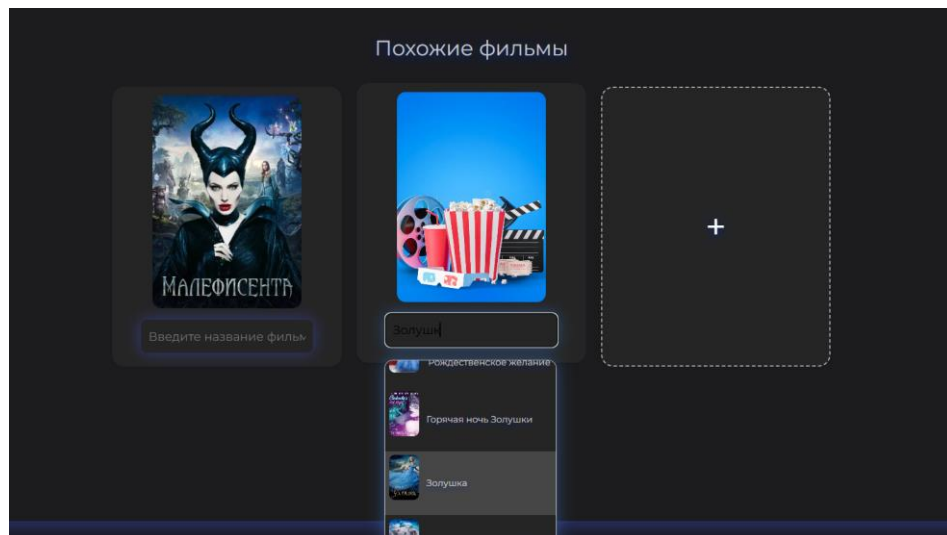


Рисунок 6.2 – Похожие фильмы



Рисунок 6.3 – Карточки с результатом рекомендаций

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ ПРИЛОЖЕНИЯ «СЕРВИС РЕКОМЕНДАЦИЙ ФИЛЬМОВ С ИСПОЛЬЗОВАНИЕМ ИИ»

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Целью разработки приложения является создание платформы, на которой пользователи смогут легко подобрать фильм исходя из собственных предпочтений. Приложение включает в себя несколько основных функций: подбор подходящей ленты на основе любимых фильмов и сохранение выбранных проектов в избранное.

Приложение будет иметь интуитивно понятный интерфейс, позволяющий пользователям быстро ориентироваться и находить нужные функции. Оно предоставит возможность поиска фильма по названию, после чего алгоритм будет осуществлять подбор схожих фильмов по жанрам, актерам, режиссерам и другим критериям, обеспечивая максимально точные рекомендации для пользователя.

Социальные функции позволят пользователям делиться фильмами и рекомендациями в социальных сетях, что обеспечит дополнительную рекламу проекта.

Веб-приложение будет доступно на различных устройствах, обеспечивая доступ к платформе из любого места через браузер. Безопасность данных пользователей будет обеспечена с помощью современных методов шифрования и аутентификации.

Целевая аудитория включает в себя людей возрастом от 12 до 60 лет в различных сферах деятельности. Существует достаточное количество рекомендательных платформ на рынке, однако данное приложение будет использовать искусственный интеллект для более точных прогнозов по предпочтениям различных пользователей, что делает его конкурентноспособным.

7.2 Расчёт инвестиций в разработку программного средства

7.2.1 Расчёт зарплат на основную заработную плату разработчиков

Расчёт затрат на основную заработную плату разработчиков производится исходя из количества людей, которые занимаются разработкой программного продукта, месячной зарплаты каждого участника процесса разработки и сложности выполняемой ими работы. Затраты на основную заработную плату рассчитаны по формуле:

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (7.1)$$

где $K_{пр}$ – коэффициент премий и иных стимулирующих выплат;
 n – категории исполнителей, занятых разработкой программного средства;

$Z_{чi}$ – часовая заработная плата исполнителя i -й категории, р;

t_i – трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Разработкой всего приложения занимается инженер-программист, Обязанности тестирования приложения лежат на инженере-тестировщике. Задачами инженера-программиста, который занимается являются создание модели данных, графического интерфейса, связи между моделью данных и графическим интерфейсом. Инженер-тестировщик занимается выявлением неработоспособных частей приложения, а также оценивает пользовательский опыт, получаемый от использования приложения.

Месячная заработная плата основана на медианных показателях для Junior инженера-программиста за 2024 год по Республике Беларусь, которая составляет примерно 900 Долларов США в месяц, а для Junior инженера-тестировщика около 600 Долларов США. По состоянию на 5 марта 2025 года, 1 Доллар США по курсу Национального Банка Республики Беларусь составляет 3,2223 Белорусских рублей.

В перерасчёте на Белорусские рубли месячные оклады для инженера-программиста и инженера-тестировщика соответственно составляют 2 900,07 и 1 933,38 Белорусских рублей соответственно.

Часовой оклад исполнителей высчитывается путём деления их месячного оклада на количество рабочих часов в месяце, то есть 160 часов.

За количество рабочих часов в месяце для инженера-программиста и инженера-тестировщика принято соответственно 196 и 32 часа.

Коэффициент премии приравнивается к единице, так как она входит сумму заработной платы. Затраты на основную заработную плату:

Таблица 7.1 – Затраты на основную заработную плату

Категория исполнителя	Меся чный оклад, р	Ча совой оклад, р	Трудоём кость работ, ч	Ит ого, р
Инженер- программист	2 900 ,07	18, 13	196	3 5 53,48
Инженер- тестировщик	1 933 ,38	12, 08	32	38 6,56
Итого				3 9 40,04
Премия и иные стимулирующие выплаты (0%)				0
Всего затраты на основную заработную плату разработчиков				3 9 40,04

7.2.2 Расчёт затрат на дополнительную заработную плату разработчиков

Расчёт затрат на дополнительную заработную плату команды разработчиков рассчитывается по формуле:

$$З_д = \frac{З_о \cdot Н_д}{100}, \quad (7.2)$$

где $Н_д$ – норматив дополнительной заработной платы.
Значение норматива дополнительной заработной платы принимает за 10 %.

7.2.3 Расчёт отчислений на социальные нужды

Расчет отчислений на социальные нужды определяется согласно ставке отчислений, которая на апрель 2023 г. равняется 35%: 29% отчисляется на пенсионное страхование, 6% – на социальное страхование. Расчёт отчислений на социальные нужды вычисляется по формуле:

$$Р_{соц} = \frac{(З_о + З_д) \cdot Н_{соц}}{100}, \quad (7.3)$$

где $Н_{соц}$ – норматив отчислений в ФСЗН.

7.2.4 Расчёт прочих расходов

Расчёт затрат на прочие расходы определяется при помощи норматива прочих расчётов. Эта величина имеет значение 30%. Расчёт прочих расходов вычисляется по формуле:

$$Р_{пр} = \frac{З_о \cdot Н_{пр}}{100}, \quad (7.4)$$

где $Н_{пр}$ – норматив прочих расходов.

7.2.5 Расчёт расходов на реализацию

Для того, чтобы рассчитать расходы на реализацию, необходимо знать норматив расходов на неё. Принимаем значение норматива равным 3%. Формула, которая использована для расчёта расходов на реализацию приведена ниже:

$$Р_p = \frac{З_о \cdot Н_p}{100}, \quad (7.5)$$

где $Н_p$ – норматив расходов на реализацию.

7.2.6 Расчёт общей суммы затрат на разработку и реализацию

Определяем общую сумму затрат на разработку и реализацию как сумму ранее вычисленных расходов: на основную заработную плату разработчиков, дополнительную заработную плату разработчиков, отчислений на социальные нужды, расходы на реализацию и прочие расходы. Значение определяется по формуле:

$$З_p = З_o + З_d + P_{\text{соц}} + P_{\text{пр}} + P_p \quad (7.6)$$

Таким образом, величина затрат на разработку программного средства высчитывается по указанной выше формуле и указана в таблице:

Таблица 7.2 – Затраты на разработку

Название статьи затрат	Формула/таблица для расчёта	Значение, р.
1 Основная заработная плата разработчиков	См. таблицу 7.1	3 940,04
2 Дополнительная заработная плата разработчиков	$З_d = \frac{3\,940,04 \cdot 10}{100}$	394,00
3 Отчисление на социальные нужды	$P_{\text{соц}} = \frac{(3\,940,04 + 394,00) \cdot 35}{100}$	1 516,91
4 Прочие расходы	$P_{\text{пр}} = \frac{3\,940,04 \cdot 30}{100}$	1 182,01
5 Расходы на реализацию	$P_p = \frac{3\,940,04 \cdot 3}{100}$	118,20
6 Общая сумма затрат на разработку и реализацию	$З_p = 3\,940,04 + 394,00 + 1\,516,91 + 1\,182,01 + 118,20$	7 151,16

7.3 Расчёт экономического эффекта от реализации программного средства на рынке

Для расчёта экономического эффекта организации-разработчика программного средства, а именно чистой прибыли, необходимо знать такие параметры как объем продаж, цену реализации и затраты на разработку.

Для оценки экономического эффекта программного продукта, предположим, что около 10 000 человек будут заинтересованы данным приложением. Из них 6 000 человек будут использовать его, а 2 500 приобретут расширенную версию. Цена на расширенную версию приложения составляет 4,99 долларов США. Таким образом, отпускная цена копии программного средства составляет 16,08 Белорусских рубля.

Для расчёта прироста чистой прибыли необходимо учесть налог на добавленную стоимость, который высчитывается по следующей формуле:

$$\text{НДС} = \frac{\text{Ц}_{\text{отп}} \cdot N \cdot \text{Н}_{\text{д.с}}}{100\% + \text{Н}_{\text{д.с}}}, \quad (7.7)$$

где N – количество копий(лицензий) программного продукта, реализуемое за год, шт.;

$\text{Ц}_{\text{отп}}$ – отпускная цена копии программного средства, р. ;

$\text{Н}_{\text{д.с}}$ – ставка налога на добавленную стоимость, %.

Ставка налога на добавленную стоимость по состоянию на 5 марта 2025 года, в соответствии с действующим законодательством Республики Беларусь, составляет 20%. Используя данное значение, посчитаем НДС:

$$\text{НДС} = \frac{16,08 \cdot 2\,500 \cdot 20\%}{100\% + 20\%}, = 6\,700,00 \text{ р.}$$

Посчитав налог на добавленную стоимость, можно рассчитать прирост чистой прибыли, которую получит разработчик от продажи программного продукта. Для этого используется формула:

$$\Delta \Pi_{\text{ч}}^{\text{р}} = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot \text{Р}_{\text{пр}} \cdot \left(1 - \frac{\text{Н}_{\text{п}}}{100}\right), \quad (7.8)$$

где N – количество копий(лицензий) программного продукта, реализуемое за год, шт.;

$\text{Ц}_{\text{отп}}$ – отпускная цена копии программного средства, р.;

НДС – сумма налога на добавленную стоимость, р.;

$\text{Н}_{\text{п}}$ – ставка налога на прибыль, %;

$\text{Р}_{\text{пр}}$ – рентабельность продаж копий.

Ставка налога на прибыль, согласно действующему законодательству, по состоянию на 5.03.2025 равна 20%. Рентабельность продаж копий взята в размере 40%. Зная ставку налога и рентабельность продаж копий (лицензий), рассчитывается прирост чистой прибыли для разработчика:

$$\Delta \Pi_{\text{ч}}^{\text{р}} = (16,08 \cdot 2\,500 - 6\,700) \cdot 40\% \cdot \left(1 - \frac{20}{100}\right) = 10\,720,00$$

7.4 Расчёт показателей экономической эффективности разработки и реализации программного средства на рынке

Для того, чтобы оценить экономическую эффективность разработки и реализации программного средства на рынке, необходимо рассмотреть результат сравнения затрат на разработку данного программного продукта, а также полученный прирост чистой прибыли за год.

Сумма затрат на разработку меньше суммы годового экономического

эффекта, поэтому можно сделать вывод, что такие инвестиции окупятся менее, чем за один год.

Таким образом, оценка экономической эффективности инвестиций производится при помощи расчёта рентабельности инвестиций (Return on Investment, ROI). Эта метрика позволяет понять, насколько выгодны вложения в разработку программного продукта и помогает принимать обоснованные решения о дальнейших инвестициях. Формула для расчёта ROI:

$$ROI = \frac{\Delta\P_{\text{ч}}^p - \text{З}_p}{\text{З}_p} \cdot 100\% \quad (7.9)$$

где $\Delta\P_{\text{ч}}^p$ – прирост чистой прибыли, полученной от реализации программного средства на рынке информационных технологий, р.;

З_p – затраты на разработку и реализацию программного средства, р.

$$ROI = \frac{10\,720,00 - 7\,151,16}{7\,151,16} \cdot 100\% = 49,91\%$$

7.5 Вывод об экономической целесообразности реализации проектного решения

Проведённые расчёты технико-экономического обоснования позволяют сделать предварительный вывод о целесообразности разработки данного программного продукта. Общая сумма затрат на разработку и реализацию составила 7 151,16 Белорусских рублей, а отпускная цена была установлена на уровне 16,08 Белорусских рублей.

Прирост чистой прибыли за год, исходя из предполагаемого объёма продаж в размере 2500 расширенных версий в год, составляет 10 720,00 Белорусских рублей. Рентабельность инвестиций за год составляет 49,91%.

Это означает, что разработка данного программного продукта является целесообразной и реализация программного средства по установленной цене имеет смысл.

Однако, следует учитывать возможные риски, связанные с конкуренцией со стороны аналогов, что может привести к незамеченности продукта на рынке. Кроме того, высокая рентабельность связана с рисками, и расчётные результаты были получены при предполагаемом объёме продаж в 2500 копий в год.

Тем не менее, при поддержке проект может получить долгосрочное и успешное развитие, и количество проданных копий может превысить предполагаемое количество.

ЗАКЛЮЧЕНИЕ

В ходе практики была выполнена значительная часть дипломного проекта, что позволило глубже понять процесс разработки программного обеспечения и его ключевые этапы. Основная цель заключалась в создании 40% дипломного проекта, что включало в себя несколько важных компонентов.

Введение дало возможность определить цели и задачи проекта, а также обосновать его актуальность. Это стало основой для дальнейшего исследования и разработки. Обзор литературы позволил изучить существующие подходы и технологии, используемые в аналогичных проектах, а также выявить потенциальные недостатки и области для улучшения.

Системное и функциональное проектирование обеспечили четкое представление о структуре и функциональности разрабатываемой системы. Были разработаны структурная схема и функциональные диаграммы, что помогло визуализировать связи между компонентами и их взаимодействие, а также определить основные функции системы.

Оформление списка литературы стало важным этапом, который продемонстрировал уровень изученности темы и источников, на которые опирался проект. Это не только подтвердило научную обоснованность работы, но и дало возможность другим исследователям ознакомиться с использованием данных ресурсов.

Разработка вводного плаката позволила кратко презентовать проект, акцентируя внимание на его ключевых аспектах и значении. Плакат стал хорошим инструментом для представления проекта широкой аудитории.

Составление экономической части дало возможность оценить финансовые затраты и выгоды, связанные с реализацией проекта. Этот аспект важен для понимания целесообразности и устойчивости разработанной системы в условиях реального рынка.

В целом, практика позволила не только реализовать значительную часть дипломного проекта, но и приобрести ценные навыки в области проектирования, разработки и управления проектами. Успешное выполнение всех этапов дало уверенность в дальнейшем развитии проекта и его завершении.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Medium [Электронный ресурс]. – The Movie Database (TMDb). – Режим доступа: <https://medium.com/@aalboughbar/the-movie-database-tmdb-a118f319ce10> – Дата доступа: 20.02.2025
- [2] TMDb [Электронный ресурс]. – Getting Started. – Режим доступа: <https://developer.themoviedb.org/reference/intro/getting-started> – Дата доступа: 20.02.2025
- [3] Хабр [Электронный ресурс]. – Коллаборативная фильтрация. – Режим доступа: <https://habr.com/ru/articles/150399/> – Дата доступа: 21.02.2025
- [4] Образование [Электронный ресурс]. – Матричная факторизация. – Режим доступа: <https://education.yandex.ru/handbook/ml/article/matrichnaya-faktorizaciya> – Дата доступа: 21.02.2025
- [5] The Clever Programmer [Электронный ресурс]. – Content-Based Filtering in Machine Learning. – Режим доступа: <https://thecleverprogrammer.com/2021/02/10/content-based-filtering-in-machine-learning/> – Дата доступа: 21.02.2025
- [6] Хабр [Электронный ресурс]. – Глубокое погружение в рекомендательную систему Netflix. – Режим доступа: <https://habr.com/ru/articles/677396/> – Дата доступа: 21.02.2025
- [7] TexTerra [Электронный ресурс]. – Рекомендации Кинопоиска. – Режим доступа: <https://texterra.ru/blog/rekomendatelnye-algoritmy-kinopoiska.html?ysclid=m8h8wk867w395375353> – Дата доступа: 15.02.2025
- [8] MovieTon [Электронный ресурс]. – Сервис быстрого подбора фильмов и сериалов. – Режим доступа: <https://movieton.org/about> – Дата доступа: 15.02.2025

ПРИЛОЖЕНИЕ А
(обязательное)

Вводный плакат. Плакат

ПРИЛОЖЕНИЕ Б
(обязательное)

Схема структурная

ПРИЛОЖЕНИЕ В
(обязательное)

Диаграмма классов